

Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Assessing distributed system dependability before deployment is hindered by absent runtime telemetry and static code analysis’s blindness to asynchronous communication topology. We present Software-as-a-Graph (SaG), a static system analysis framework that constructs typed multigraphs from Architecture-as-Code manifests across five entity types. SaG pairs two parameter-independent pathways: a relation-specific Heterogeneous Graph Transformer (HGT) with Quality-of-Service (QoS) edge encodings to forecast cascading failure blast radii, and an interpretable ISO/IEC 25010 layer attributing fragility to Availability (single points of failure) or Fault Tolerance (cascade hubs) to guide targeted repairs.

Across twelve synthetic scenarios and five open-source reference systems: (1) relation typing provides a small but fold-consistent gain over homogeneous learning under inductive distribution shift ($\Delta\rho = +0.082$, winning all twelve folds, $p = 0.0005$), replicated by the unweighted pair (+0.064, 12/12); (2) against a training-free QoS-weighted centrality baseline, the learned model achieves parity without statistical superiority (+0.058, $p = 0.204$ fixed; +0.115, $p = 0.052$ nested); (3) on authentic open-source systems, training-free heuristics outperform zero-shot neural transfer ($\rho \approx 0.51$ vs. 0.125), while the model’s prediction spread reliably detects this out-of-distribution transfer gap at inference time without labels; (4) an HGT forward pass requires only 43.7 ms at 2,000 components, with the complete gate executing in 0.02–27.4 s. Graph learning is therefore justified by relational attention, multi-hop relationship criticality, standards-grounded root-cause attribution, and inference-time self-diagnosis, delivering a computationally sustainable, zero-telemetry dependability gate for continuous integration.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Distributed systems dependability, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish-subscribe (pub-sub) architectures. Across diverse domains—from autonomous driving (ROS [1]) and enterprise event streams (Apache Kafka [2]) to cyber-physical backbones (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters—pub-sub decouples producers and consumers in space, time, and synchronization [7]. Components interact indirectly through intermediate message topics and brokers without maintaining direct static references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—such as reliability guarantees, durability, message priorities, and delivery deadlines—to govern how traffic behaves under peak load and network stress.

While this architectural decoupling confers elastic scalability and operational flexibility, it creates a formidable **visibility barrier** for system performance, reliability, and computational sustainability:

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

- **Indirect Failure and Degradation Pathways:** In traditional synchronous architectures (e.g., RESTful HTTP or gRPC), component interactions follow explicit caller–callee invocation paths. In asynchronous pub-sub and event meshes, publishers and subscribers share no direct references. Cascading failures, queue head-of-line blocking, and backpressure propagate across hidden logical paths spanning brokers, shared topics, colocated execution nodes, and shared libraries.
- **Distinct Degradation Mechanisms:** Disturbances in complex distributed systems do not propagate in a uniform manner. They manifest either as *sequential cascades* (e.g., a slow subscriber causing broker queue saturation and upstream backpressure) or as *simultaneous blast radii* (e.g., a shared runtime library crash, memory exhaustion, or host machine outage instantly disabling multiple colocated services). Conventional architectural diagrams and static call graphs fail to represent these multi-layer dependencies.

Addressing these architectural vulnerabilities is most effective and cost-efficient **prior to deployment**, during design and Continuous Integration / Continuous Delivery (CI/CD), adhering to established foundational principles of dependable computing [8]. However, at design and build time, **no runtime telemetry, distributed tracing, or operational logs exist**. Consequently, software architects, performance engineers, and Site Reliability Engineers (SREs) face two fundamental questions without operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural repair (such as replicating a message broker, decoupling an over-subscribed topic, or sandboxing a shared library) will most effectively eliminate that risk?*

The same questions are paramount for system performance and computational sustainability. In production, unmitigated cascading failures trigger catastrophic restart storms, queue head-of-line blocking, and connection-pool thrashing that exhaust CPU cycles and cloud infrastructure budgets on uncompleted work. Hardening distributed topologies at design time eliminates these systemic vulnerabilities at the lowest possible cost. Furthermore, from a sustainable software engineering perspective, SaG provides an ultra-lightweight Static System Analysis gate that executes in seconds (with a 43.7 ms neural forward pass), obviating the massive compute, memory, and energy footprint required to provision and run multi-node dynamic chaos testing clusters during continuous integration (Sections 7.5 and 8.2). While this study predicts topological failure impact and measures analysis latency rather than physical energy counters (Joules) or runtime throughput, its primary sustainability contribution lies in delivering a green, compute-efficient CI/CD gating mechanism.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** We forecast dynamic cascading failure blast radii and identify critical components using a data-driven, relation-specific model over learned topological representations. Closed-form topological metrics capture broad connectivity cheaply; whether they also resolve multi-hop, relation-dependent cascade spread across heterogeneous channels is an empirical question, which we test directly against such a baseline (Section 7.1). Our predictive pathway is trained and evaluated against independent simulation ground truth as a ranking and critical-set identification model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist indicates *where* risk lies, but not *how to fix it*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [9] and ISO/IEC 25019 [10]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing, for instance, an unreplicated single point of failure from a high-coupling maintainability bottleneck—to guide concrete repairs. It serves strictly as an attribution model, not a ranking model.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other’s output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this independence allows SaG to identify components that are structurally central yet operationally low-impact—a nuanced diagnosis unattainable by either pathway alone.

Existing software engineering approaches fail to bridge what we define as the **Architecture–Code Gap**: *a distributed system can have pristine, bug-free source code within each individual service, yet remain fragile to catastrophic global outages caused by hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts*. Although classical architecture evaluation methods such as the Architecture Tradeoff Analysis Method (ATAM) [11, 12] identify architectural risks, and software studies analyze architectural technical debt [13] and bad smells [14], they rely on manual stakeholder elicitation rather than quantitative structural learning. Prevailing automated paradigms each leave part of this gap unaddressed. Static code analysis [15, 16, 17, 18] inspects complexity, cohesion and coupling *within* a service but cannot observe message queues or cross-host propagation. Runtime chaos engineering [19] injects real faults but needs a provisioned cluster, carries operational risk, and arrives after the architecture is fixed. Homogeneous centrality metrics [20, 21, 22, 23] flatten the system into an untyped graph in which a message topic, a shared library and an execution host are indistinguishable. Section 2 develops each of these in turn.

Furthermore, while machine learning has demonstrated remarkable success across software engineering, contemporary AI approaches applied to system dependability often function as uninterpretable black boxes. Deep neural models frequently output scalar risk scores or latent embeddings without providing transparent, actionable rationales for their predictions. In mission-critical software engineering, an opaque risk score is inadequate: developers and SREs cannot refactor code or reconfigure infrastructure without understanding *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming the black-box AI challenge, this work introduces **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and outputs per-relationship criticality alongside multi-task quality outputs (Section 4).
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), pointing to distinct repairs. Because it is a linear, propagation-free aggregate by design, it explains *why* a component is vulnerable rather than how far a cascade travels; its standalone rank correlation is correspondingly modest (Section 7.1).

To ensure methodological rigor, SaG enforces a strict **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical graph G_{analysis} , while ground-truth failure impacts are generated by independent discrete-event simulators operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4).

Figure 1 shows how the two pathways relate. The predictive pathway is the primary one and the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not — and that oracle is strictly an offline training-and-validation component, never a dependency of online inference. The explanation layer then characterises what the predictor flagged and the remediation that implies. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor ranked. The remediation guidance closes a loop of its own, in which each candidate edit is re-simulated on its own mutated copy of $G_{\text{structural}}$ and kept only if it beats the simulator’s seed-to-seed noise.

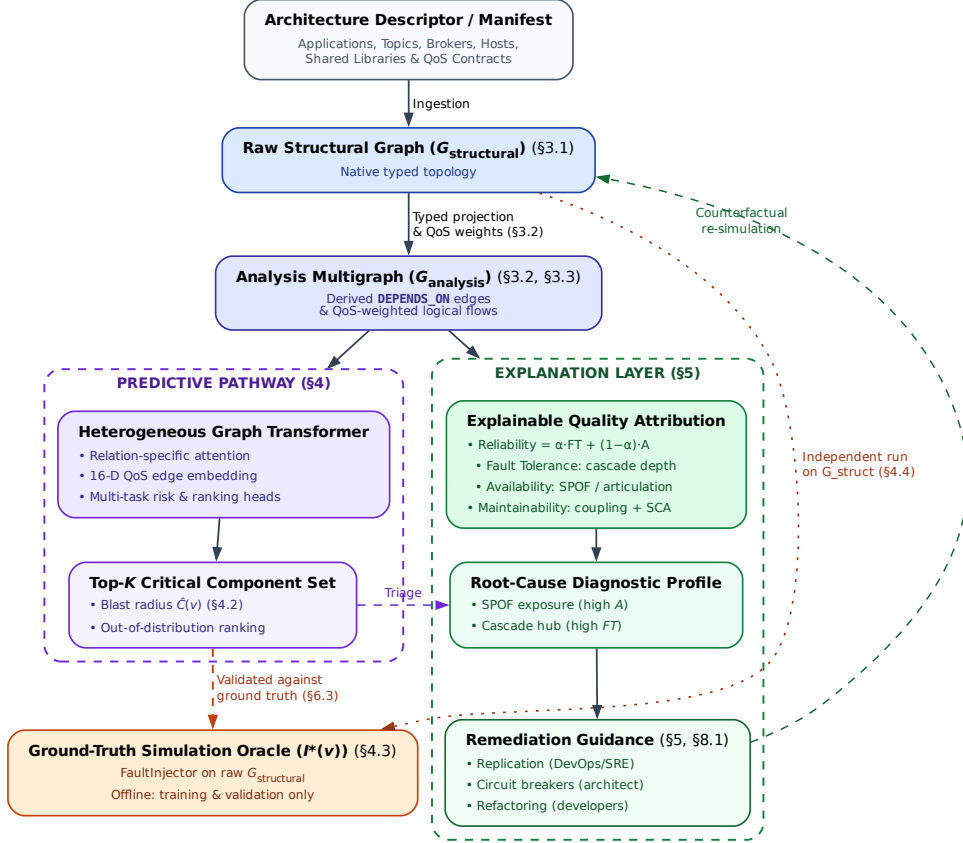


Figure 1: End-to-end architecture of the SaG framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted **DEPENDS_ON** projection → typed node features) feeds two pathways that share no parameters: the predictive pathway (Section 4), which emits a ranked critical set and per-relationship criticality, and the explanation layer (Section 5), which emits a standards-grounded quality profile. The simulation oracle scores only the former and runs on $G_{\text{structural}}$ alone.

Rationale for Graph Learning vs. Direct Simulation. Since discrete-event simulation $I^*(v)$ defines ground-truth criticality here, it is fair to ask why train a graph model at all rather than run simulation sweeps or closed-form heuristics. Four practical reasons motivate the design, and Section 7.1 tests the last of them adversarially. Message passing generalizes across labelled and unlabelled entities alike, which mattered more when it was written than it does now: the injector originally reached only active application processes, leaving 30–47% of components per system (topics, hosts) without direct labels, and it has since been extended to express Topic and host-Node failure directly (Section 8.3). We record the change rather than retain the

argument, because the coverage gap it appealed to is closed on this corpus; what survives is the weaker claim that a trained model still scores entity types no simulator sweep was run for. Cascade simulation is stochastic and seed-sensitive (label standard deviation reaches 0.416), while a trained model learns a smooth, threshold-marginalized surrogate that re-scores an already-analysed architecture cheaply. Neither a simulator nor an unaugmented GNN returns a root cause in standardized quality terms: knowing that a subscriber lost its feed does not distinguish an unreplicated single point of failure from an error-cascade hub, and the two demand different repairs. Finally, dynamic simulators need runnable containers or communication harnesses, whereas graph learning scores Architecture-as-Code manifests before any runtime infrastructure exists. Whether these motivations are borne out empirically is a separate question, and Section 7.1 answers it only partly in the framework’s favour.

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding, Calibration and Sensitivity): *How do middleware Quality-of-Service policies, the declared weighting constants of the explanation layer, the choice of simulation oracle, and propagation-threshold and normalization settings affect forecasting performance, stability, and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2), cloud-native microservices, smart home IoT, and industrial edge computing?*

RQ5 (Analysis Cost and Computational Sustainability): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that computational footprint, and does the resulting budget enable sustainable, per-commit architectural gating?*

1.5. Key Contributions

This paper presents four principal contributions:

- Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code alone, with a 16-dimensional edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Our central empirical claim concerns cross-architecture transfer: with matched training sets, depth and model selection, typed learning leads to untyped learning by $\Delta\rho = +0.082$ (0.659 vs. 0.577) under inductive distribution shift, winning all 12 folds ($p = 0.0005$; Section 7.2), and the unweighted pair replicates this exactly ($+0.064$, 12/12, $p = 0.0005$). The effect is small — a third of the QoS-weighted baseline’s own $+0.253$ over unweighted centrality — but it is the only comparison in this study that wins every fold, and it holds in-distribution as well ($+0.088$, 6/7, $p = 0.047$), so relational typing is a consistent small benefit in both regimes rather than a purely inductive bias. Because the homogeneous baseline receives the scalar QoS aggregate rather than the full 16-D encoding, this margin could in principle combine typing with per-dimension QoS encoding; an ablation within the typed architecture attributes none of it to the QoS dimensions (-0.005 , $p = 0.68$; Section 7.3.1). Simultaneously, we establish the honest empirical boundary: against an unparameterized QoS-weighted centrality baseline (Topo-QoS), out-of-distribution ranking is not surpassed — $\Delta\rho = +0.058$ ($p = 0.204$) at fixed configuration and $+0.115$ ($p = 0.052$) under nested selection — and the critical-set margin ($F_1@K = 0.439$ vs. 0.375) is not statistically significant either, won in 7 of 12 folds ($p = 0.23$; Section 7.1).

2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures, supplying the typed substrate the predictor consumes (Section 3).
3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns the predictor’s ranked output into an actionable diagnosis, separating single-point-of-failure exposure from error-propagation depth—two distinct failure modes requiring different repairs (Section 5).
4. **Empirical Benchmark, Real-World Evaluation, and Cost Characterization:** An evaluation across eleven synthetic topologies (2,387 components) and five open-source reference systems (351 components) under strict graph-view separation, establishing both where typed graph learning helps and the boundary where it does not, with a per-stage cost profile locating the pipeline’s cost in its deterministic graph-analysis stage rather than in the learned model (Sections 6–7).

Relationship to the authors’ prior work. An earlier, shorter version was presented at a peer-reviewed conference [24], covering the preliminary typed multigraph formulation and the deterministic quality-attribution model on the synthetic corpus alone. This manuscript is a substantially extended version meeting the JSS extension policy. New here are: the entire predictive pathway (the HGT, its 16-D QoS edge encoding, the multi-task masked-loss heads, and every learned result in Section 7); the inductive LOSO protocol and cross-architecture transfer analysis (Section 7.2); zero-shot evaluation on five open-source reference systems (Section 7.4); the cost characterization answering RQ5 (Section 7.5); the four-oracle convergent-validity taxonomy and graph-view separation (Sections 4.3–4.4); global sensitivity analysis over all ten weight constants (Section 7.3); and the homogeneous-versus-heterogeneous comparison under substrate parity (Section 7.2). Material retained from the conference version is limited to preliminary formalisms in Sections 3 and 5, both restructured and expanded. No companion manuscript from this work is under consideration elsewhere.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work on distributed systems dependability, performance engineering, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model, the dependency projection rules, and the typed node features consumed by both pathways. Section 4 presents the Heterogeneous Graph Transformer, its multi-task heads, the simulation oracles that supply its labels, and the input–label independence guarantee. Section 5 introduces the interpretable RM explanation layer. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ5. Section 8 discusses architectural implications, performance and computational sustainability, threats to validity, limitations, and concluding remarks.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [19] inject faults or latency into staging or production clusters to observe degradation and recovery. This delivers operational validation that no static method can match, but it requires a fully provisioned cluster, carries the risk of real service disruption, and consumes substantial cluster time per sweep — so it cannot be applied during architectural design or early CI/CD.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned, enabling architectural gating within CI/CD pipelines without a running cluster.

Architecture-Based Reliability Prediction. Predicting dependability from an architectural description before deployment is not a new ambition, and SaG should be read against the tradition that pursued it analytically. Cheung’s absorbing-Markov-chain model [25] derives system reliability from component reliabilities and a transfer-of-control graph; Goseva-Popstojanova and Trivedi [26] systematize the state-based, path-based and additive families that followed, and Immonen and Niemelä [27] survey the resulting methods from the architectural perspective. Model-driven descendants such as the Palladio Component Model [28] and layered queueing networks [29] predict performance and reliability from parameterized component models with well-understood solution techniques.

These methods are complementary to ours rather than superseded by it, and the distinction is one of required inputs rather than of accuracy. They need per-component failure probabilities, transition probabilities or service demands — parameters that are themselves estimated from operational profiles, measurement or expert judgement, and that are unavailable in the setting we target (a manifest at commit time, with no telemetry). SaG asks a narrower question in exchange: not what the system’s reliability *is*, but which components’ failures would propagate furthest through the declared topology. Where those parameters can be obtained, an analytical model answers a stronger question than a ranking does, and we make no claim to displace it.

Data-Driven Failure Prediction and Root-Cause Analysis in Microservices. A large recent literature localizes faults in microservice systems from operational data. Seer [30] and Sage [31] predict and debug QoS violations from traces and hardware telemetry; MicroRCA [32] and TraceRCA [33] localize root causes over service-dependency and trace graphs; DeepTraLog [34] and Eadro [35] combine traces, logs and metrics under graph-based deep models. This line is the closest methodological neighbour to our predictive pathway, and it consistently outperforms what a purely static analysis can achieve — because it observes the running system. That is precisely the boundary: every one of these approaches requires a deployed system emitting traces, logs or metrics, and therefore cannot answer a question posed at design or pull-request time. SaG occupies the pre-deployment complement, and accepts a correspondingly weaker evidential basis: simulated rather than observed failures, and topology rather than behaviour.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [15]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [16], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [17, 18], and code duplication to flag internal code smells and defect-prone modules [36, 37, 38, 39]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

To bridge this “Architecture–Code Gap,” **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [40, 41] and architectural technical debt [42] early during continuous integration (CI/CD) [43, 44], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [9] and the **ISO/IEC 25019:2023** Quality-in-Use model [10]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Faultlessness, Availability, Fault Tolerance and Recoverability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Reusability, Analysability, Modifiability and Testability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

SaG operationalizes a strict subset of these: Availability and Fault Tolerance under Reliability, and Modularity, Modifiability and Analysability under Maintainability (Section 5.1). Faultlessness, Recoverability, Reusability and Testability are not derivable from deployment topology alone and are outside the scope of this work.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [45, 46]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [47] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure mathematical soundness in weighting models. This study applies AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, in conjunction with learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [20, 22], articulation points, and PageRank [21, 23]. Foundational studies on network robustness [48], cascading overloads [49], and interdependent networks [50] model how disruptions propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality scalar cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating cascade hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared library, and a physical execution host.

To overcome the limits of hand-engineered metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [51], DrBC [52], and PowerGraph [53]). However, most available models rely on **homogeneous message passing** (GCN [54], GraphSAGE [55], GAT [56]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries and fail to generalize out-of-distribution.

Heterogeneous Graph Neural Networks (RGCN [57], HAN [58], HGT [59], MAGNN [60]) resolve this by employing relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [59] to preserve typed relational semantics when forecasting cascading failure blast radii and performance degradation.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [61] and PGExplainer [62], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \quad (2)$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \rightarrow [0, 1]$ and $w_E : E \rightarrow [0, 1]$ are weighting functions representing entity criticality and connection strength.

Table 1 summarizes the five entity types and six structural edge types formalized in the SaG model, along with their semantics and representative concrete distributed systems implementations.

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a **RELIABLE** topic with **TRANSIENT_LOCAL** durability binds communicating services substantially more tightly than a **BEST_EFFORT** telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75, 0.15, 0.10) \quad (3)$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (4)$$

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	<code>librdkafka</code> , OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Node	Process is hosted on physical/virtual host
CONNECTS_TO	Node $\rightarrow$ Node	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and transport priority both govern in-flight delivery quality, with reliability receiving higher weight because unconditional delivery guarantees precede message scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix with a small, non-zero consistency ratio ($CR \approx 0.016 \leq 0.10$). The modulators are logarithmically compressed and clamped to $[0, 1]$: $\text{SizeNorm}(t) = \min(1.0, \log_2(1 + \text{bytes})/20)$ (a 1 MiB design envelope, representing the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \min(1.0, \log_{10}(1 + \text{Hz})/3)$. The final weight $w(t)$ is clamped to $[0.01, 1]$, ensuring that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES) inherits $w_E(e) = w(t)$ along with the topic’s QoS vector.

The outer split (β, α, ψ) is a declared convex combination whose sensitivity is evaluated in Section 7.3.2.

3.2.1. Logical Dependency Projection (*DEPENDS_ON*)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, *DEPENDS_ON*, directed from *dependent* to *dependency* (“if target fails, source is impacted”), according to the six projection rules detailed in Table 2:

Table 2: The six *DEPENDS_ON* logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	<code>app_to_app</code>	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	<code>app_to_broker</code>	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	<code>node_to_node</code>	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	Lifted max w
4	<code>node_to_broker</code>	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	Lifted max w
5	<code>app_to_lib</code>	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	<code>broker_to_broker</code>	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{node})$

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [63, 64, 65]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [66] to combine the consuming Application’s and the shared Library’s vertex weights, balancing caller and dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It does not imply that one broker functionally depends on another, but rather that two brokers colocated on the same host share that host’s physical failure domain. This follows the same simultaneous-blast principle as Rule 5, which is why the derived weight equals the shared Node’s weight and the relation is bidirectional. In production middleware deployments, colocated brokers compete for host resources (CPU cores, page cache, file descriptors, and NIC bandwidth); a host outage takes down all colocated instances simultaneously. Operational best practices for Kafka, RabbitMQ, and EMQX recommend distributing brokers across fault domains. Rule 6 does not model directional intra-cluster broker coupling (e.g., partition replication, controller quorum election, federation, or shovel links), which do not require physical colocation; extending the schema to capture these interactions is reserved for future work. In our evaluation corpus, Rule 6 applies in four of the eight cached scenarios (the seven synthetic topologies of Table 4 plus the ATM case study of Section 6.1) and contributes only 12 directed edges in total. Because the simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as PUBLISHES_TO, ROUTES, RUNS_ON, and USES). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived DEPENDS_ON edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis} .

Figure 2 illustrates this duality on a running example, contrasting the raw structural graph against the derived DEPENDS_ON projection.

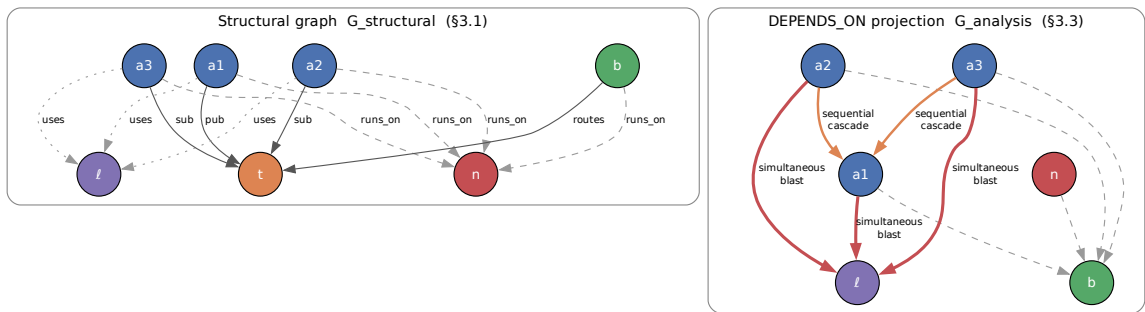


Figure 2: Running example: the raw structural graph (left) and the DEPENDS_ON projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [67].

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis} : the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load) produced by the deterministic analysis stage whose cost is characterized in Section 7.5. Type-specific features extend that block:

- **Application (23 dims):** indices 18–22 add source-code metrics from SCA — lines of code, cyclomatic complexity, Martin’s instability $I_{\text{code}} = C_e / (C_a + C_e)$ [68], Lack of Cohesion in Methods, and the composite Code Quality Penalty (CQP).
- **Library (25 dims):** the Application block plus two library-specific blast-radius drivers (23–24): the normalized size of the transitive reverse-USES closure, and the normalized count of distinct subscribers reachable from topics published within that closure.
- **Broker (19 dims):** index 18 is normalized queue buffer capacity.
- **Topic (22 dims):** indices 18–21 are publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node (20 dims):** indices 18–19 are normalized CPU core allocation and physical memory.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on neighbor count, but through architectural relations and dependencies extending multiple hops beyond the initial fault. Whether a closed-form combination of standard centrality metrics can capture these compound dynamics is an empirical question rather than a settled one. The primary predictive pathway of Section 1.2 therefore employs a learned graph model, and Section 7.1 evaluates it against exactly such a closed-form baseline — which, on out-of-distribution ranking, it does not significantly surpass.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input-label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [59], implemented within PyTorch Geometric [69], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e in G_{analysis} ; indices 2–8 one-hot encode the seven structural and derived relations (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON). Indices 9–15 encode middleware QoS parameters on PUBLISHES_TO and SUBSCRIBES_TO edges (zeroed elsewhere). Four of these dimensions are active in our evaluation corpus:

reliability (0 best-effort, 1 reliable); durability (0 volatile, 0.5 transient-local, 0.6 transient, 1 persistent); message priority (0, 0.33, 0.66, 1); and a heterogeneity flag set when the edge’s QoS triple departs from its scenario’s modal profile. The remaining three indices (deadline-active flag, $\log_{10}(1 + \text{deadline_ns}/10^6)$, and $\log_{10}(1 + \text{max_blocking_ms})$) are architectural schema provisions designed to accommodate hard real-time DDS and ROS 2 microsecond profiles, and are zero-initialized in the current benchmarks.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D -dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v)) \quad (5)$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention across H heads:

$$\text{Attn}(u, e, v) = \text{Softmax}_{\forall u \in \mathcal{N}(v)} \left(\frac{K(u)W_{\text{att}, \phi(e)}Q(\tilde{h}_v)^\top}{\sqrt{D/H}} \right) \quad (6)$$

$$\text{Msg}(u, e, v) = V(u)W_{\text{msg}, \phi(e)} \quad (7)$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream back-pressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$).
4. **Residual Aggregation and Layer Normalization:** Target node representations are updated across layers $l \in \{1, \dots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \quad (8)$$

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4} , and dropout probability $p = 0.10$ applied post-attention. Learning rates follow a cosine annealing schedule with warm restarts (CosineAnnealingWarmRestarts, $T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\text{min}} = 3 \times 10^{-6}$). Training executes for a maximum of 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss over labeled nodes. Inductive subgraphs are processed per scenario using full-graph inductive packing without mini-batch subsampling, with validation masks isolating held-out nodes to prevent information leakage across data splits. Five independent random seeds {42, 123, 456, 789, 2024} are evaluated across all runs, redrawing both partition masks and initializations. *Selection protocol:* the architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate and schedule) and the loss coefficients of Equation (9) were fixed a priori from values conventional for HGT [59] and were not tuned against any evaluation reported in this paper; no search over them was performed, on either the in-distribution test split or the LOSO folds. This avoids selection leakage, at the cost of leaving open whether either family is reported near its own optimum — a comparison between untuned configurations, which we state rather than treat as a like-for-like optimum comparison.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \quad (9)$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE ranking loss [70], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}})$ regresses predicted heads toward the diagnostic pathway’s baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via **FaultInjector**) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \quad (10)$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

4.2.2. Domain-Rewighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \quad (11)$$

where $M_{\text{static}}(v)$ is obtained directly from the structural analyzer’s maintainability baseline, combining learned dynamic reliability with static source-code maintainability. Because maintainability is unobserved in dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly, while domain reweighting sensitivity is evaluated against the static RM baseline in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via **FaultInjector**, it crashes component v , propagates outages across dependent topics, brokers and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population, each subscriber contributing the unweighted mean loss of the topics it subscribes to. The denominator is the full set of subscribers in the intact graph: subscribers that themselves fail during the cascade are retained in the average rather than excluded from it. A topic’s feed loss is the fraction of its publishers that have failed (for a topic with no publisher, the fraction of its failed routers), scaled by a QoS ladder ($\times 1.2$ **RELIABLE**, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. The implementation admits a per-publisher rate weighting, but publication rates are declared per topic rather than per publisher throughout our corpus, so every publisher of a topic carries the same rate and the rate-weighted form reduces exactly to this publisher fraction. This is the **primary continuous target label** throughout.
- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$):** Implemented via **FailureSimulator**, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta \text{Reachability} + 0.25 \cdot \Delta \text{Fragmentation} + 0.25 \cdot \Delta \text{Throughput} + 0.15 \cdot \Delta \text{FlowDisruption} \quad (12)$$

where each term is weighted by operational severity $s(t) = w(t) \cdot \text{rate}(t)$. $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.

Reading the rate modulator. $\text{rate}(t)$ is the topic’s observed message rate, read from a runtime telemetry profile when one is attached and defaulting to unity otherwise. No such profile is attached in any evaluation reported in this paper, which draws its ground truth from structural models rather than production observation; $s(t) = w(t)$ throughout Section 7, and the rate channel should be read as an interface for deployment-time calibration rather than as an active term in our results. Publication frequency and payload size nonetheless reach $s(t)$, through the FreqNorm and SizeNorm modulators internal to $w(t)$ (Section 3.2). The severity term is therefore a function of declared QoS semantics and objective physical topic attributes only.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy framework [71], this oracle simulates message emission rates, stochastic network latencies, broker buffer saturation, and queue drops under fault injection. It extracts the drop in delivered message rate suffered by surviving consumers.

- **Change-Propagation Oracle ($I_M(v)$):** Implemented via `ChangePropagationSimulator`, this oracle executes a deterministic breadth-first traversal of the reversed dependency graph to quantify maintenance change impact:

$$I_M(v) = 0.45 \cdot \text{ChangeReach}(v) + 0.35 \cdot \text{WeightedChangeImpact}(v) + 0.20 \cdot \text{NormalizedChangeDepth}(v) \quad (13)$$

- **Relationship (Edge) Removal Oracle ($I_{\text{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping endpoint components operational. Writing $\bar{I}_{\text{comp}}(G) = |V|^{-1} \sum_{v \in V} I_{\text{comp}}(v; G)$ for the mean composite impact over a graph G :

$$I_{\text{edge}}(u, v) = \bar{I}_{\text{comp}}(G \setminus \{(u, v)\}) - \bar{I}_{\text{comp}}(G) \quad (14)$$

Topic Criticality Label Masking. `FailureSimulator` can blend a declared `Topic.criticality` into its severity term; this is disabled, because that field is a GNN input feature (`topic_qos_criticality_ord`) and consuming it would score the predictor against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^* , I_{comp} , I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ (**FaultInjector**) as the **primary oracle** for all predictive ranking results (Tables 6–8, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe, and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across seven benchmark scenarios and five random seeds on the Application population, the mean Spearman rank correlation is $\rho = 0.907$ for (I_{dyn}, I^*) , $\rho = 0.425$ for (I_{comp}, I^*) , and $\rho = 0.427$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The strong rank agreement ($\rho = 0.907$) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms. However, agreement on the top- K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.49 for the strongest pair and 0.24–0.28 for the two I_{comp} pairs, vs. 0.111 expected by chance), highlighting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

4.4. Input-Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.

- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardized quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck. These diagnoses call for distinct architectural repairs: replicating the host or broker, decoupling the topic, or refactoring the module. This section presents the layer supplying that diagnosis. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [9], **ISO/IEC 25019:2023** (Quality-in-Use) [10], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [72], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system’s capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. The two are separated because they imply different repairs, not because they predict different runtime quantities: this layer attributes structural fragility and does not forecast latency, throughput or queue occupancy. Table 3 outlines this Reliability–Maintainability (RM) quality decomposition, mapping ISO/IEC sub-characteristics to architectural questions, underlying graph metrics, and targeted engineering remediations.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	Fault Tolerance (FT)	How broadly does failure propagate?	Reverse PageRank on G^T , in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	Modularity/Modifiability	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 Automotive Safety Integrity Level [ASIL] ratings) and Security (which requires explicit threat models, such as STRIDE [Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege]) fall outside purely structural topology analysis and are reserved for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [47]:

1. **Fault Tolerance** ($FT(v)$): Measures error cascade potential on the transpose graph G_{analysis}^T (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \quad (15)$$

where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v)$ is the enhanced Cascade Depth Potential term.

2. **Availability** ($A(v)$): Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component’s own QoS weight:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w(v) \quad (16)$$

where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\text{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is the component’s intrinsic QoS weight.

3. **Reliability** ($R(v)$): Blends Fault Tolerance and Availability hierarchically:

$$R(v) = r_\alpha \cdot FT(v) + (1 - r_\alpha) \cdot A(v), \quad r_\alpha = 0.36 \quad (17)$$

The blend weight is written r_α throughout to distinguish it from the payload-size coefficient α of Equation (3); the two are unrelated parameters and are screened separately in Table 11. Intra-dimension pairwise comparison matrices are audited against Saaty’s consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights apply a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports ranking sensitivity to λ).

4. **Maintainability** ($M(v)$): Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \quad (18)$$

where $\text{BT}(v)$ is Betweenness Centrality, $w_{\text{out}}(v)$ is QoS-weighted efferent coupling (out-degree), $\text{CQP}(v)$ is the Code Quality Penalty, $\text{CouplingRisk}_{\text{enh}}(v)$ is an afferent/efferent coupling-risk imbalance term, and $\text{CC}(v)$ is the local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (19)$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^T$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \quad (20)$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This yields a directly actionable distinction: a component scoring high on A but low on FT is a single point of failure calling for horizontal replication, whereas one scoring high on FT is an error-cascade hub calling for circuit breakers, rate limiting and bulkhead isolation. Whether acting on these diagnoses improves any measured runtime property is not evaluated here (Section 8.4).

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 1,896 components distributed across twelve distinct system architectures, as detailed in Table 4:

Table 4: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$ V $	$ V_{app} $	Topics	Brokers	Hosts	Libs	$ E $
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [73]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [74]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [75]	Real-World Microservices	90	41	30	3	8	8	162
Home Assistant [76]	Real-World Smart Home	63	24	22	3	6	8	119
EdgeX Foundry [77]	Real-World Industrial IoT	63	22	24	3	6	8	112
Total		1,896						

Here, $|V|$ is the sum of all five entity-type counts per scenario, totaling 1,896 components across the primary benchmark suite (1,545 components across the seven core synthetic domains, and 351 components across the five real-world systems). $|E|$ counts every raw structural relationship instance recorded in the scenario specification (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES) — the native substrate that simulation oracles traverse — rather than the derived DEPENDS_ON projection constructed for GNN training.

Five real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The synthetic scenarios were produced by a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, minimum, maximum, Q_1 , Q_3) for application publish and subscribe fan-out, applications per host, library fan-in, topic payload size, and categorical distributions over the three QoS dimensions. Graph degree distributions and clustering emerge directly from these parameters rather than being synthetically forced. Table 5 reports the generative parameters governing each topology’s shape; complete configurations are included in the replication package.

Table 5: Generative parameters of the eleven synthetic evaluation scenarios. Counts, seed and fan-out figures are read directly from the committed configurations. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config’s declared QoS targets, which domain-driven assignment does not always realize (Section 6.1).

Scenario	Config	Seed	Counts	Pub	Sub	Modal QoS (R/D/P)
Autonomous Vehicle (AV)	scenario_01_autonomous_vehicle	1001	80/40/4/8/20	2.5	5.0	RELIABLE/TRANSIENT_LOCAL/HIGH (85–100%)
Enterprise Pub-Sub	scenario_07_enterprise_xlarge	7007	300/120/10/40/50	3.0	4.5	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Financial Trading	scenario_03_financial_trading	3003	60/35/5/6/18	4.0	6.0	RELIABLE/PERSISTENT/CRITICAL (51–83%)
Healthcare Integration	scenario_04_healthcare	4004	50/25/3/8/12	2.5	3.0	RELIABLE/PERSISTENT/MEDIUM (60–76%)
Hub-and-Spoke	scenario_05_hub_and_spoke	5005	70/30/2/12/25	2.0	7.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Industrial SCADA	scenario_19_industrial_scada	1902	140/70/4/25/15	1.1	1.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
IoT Smart City	scenario_02_iiot_smart_city	2002	200/80/6/30/10	2.0	1.5	BEST_EFFORT/VOLATILE/LOW (56–79%)
Logistics Fleet	scenario_21_logistics_fleet	2104	110/50/7/18/20	1.8	1.9	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Microservices Mesh	scenario_06_microservices	6006	90/45/6/15/30	1.5	2.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Real-Time Gaming	scenario_20_realtime_gaming	2003	75/38/5/12/28	2.6	2.8	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Telecom RAN	scenario_18_telecom_ran	1801	120/55/8/20/22	2.2	1.6	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)

Corpus Composition across Experimental Regimes. The experimental evaluation spans three complementary regimes with precisely bounded corpora:

1. *In-Distribution Evaluation (Table 6)*: Evaluated on the seven core synthetic domains from Table 4 using stratified 60% train / 20% validation / 20% test node splits over five random seeds.

2. *Inductive Leave-One-Scenario-Out (LOSO) Cross-Validation (Table 8)*: Evaluated across twelve distinct inductive folds totaling 2,387 components: the seven core synthetic scenarios, four extended domain topologies (Telecom RAN, Industrial SCADA, Realtime Gaming, and Logistics Fleet), and an Air Traffic Management (ATM) network scenario. In each fold, models are trained on eleven graphs and tested zero-shot on the held-out twelfth graph.
3. *Real-World Architectural Transfer (Tables 13 and 14)*: The five open-source real-world systems (Auto-ware.universe, Cloud Microservices, Train-Ticket, Home Assistant, EdgeX Foundry) are never used as training folds; they are withheld entirely and used strictly for zero-shot architectural transfer validation.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than merely statically archived. Each dataset is deterministically generated from its configuration file via:

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset regenerates *byte-identically* from its configuration and that all disk digests match the manifest. This guarantees that third parties can reproduce the exact graphs used in our experiments, rather than simply sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations drawn from three families. Predictor names state the family and the substrate: an `-N` suffix marks a model trained on the *native* multigraph, its absence the derived Application–Library flow projection, and a `-QoS` suffix marks a configuration that consumes declared QoS contracts. *SaG* throughout denotes the framework, never an individual predictor.

1. **Heterogeneous graph learning (typed HGT). HGT-QoS** (proposed): relation-specific Heterogeneous Graph Transformer (Section 4) ingesting the complete native multigraph with 16-dimensional continuous-categorical edge features that encode middleware QoS contracts. Its ablation **HGT**, which masks those QoS dimensions, is reported in Section 7.3.1.
2. **Homogeneous graph learning (untyped GAT). GAT-N-QoS**: homogeneous Graph Attention Network [56] trained on the identical native multigraph substrate with per-type input projections, but untyped, single-relation message passing. Its edge channel carries the scalar QoS aggregate $w(e)$ — dimension 0 of the same 16-D encoding HGT-QoS consumes — rather than the per-dimension decomposition; no homogeneous architecture in our suite ingests the full 16-D vector. The HGT-QoS–GAT-N-QoS contrast therefore bounds the *joint* contribution of relational typing and per-dimension QoS encoding. Section 7.3.1 separates the second factor within the typed architecture, and the corresponding unweighted ablation is **GAT-N**.
3. **Structural baselines (training-free). Topo-QoS**: QoS-weighted topological centrality evaluated on the derived application flow projection.
4. **Structural baselines (training-free). Topo**: structural centrality combining unweighted betweenness centrality and articulation point scoring on the flow projection.

In addition, the out-of-distribution evaluation (Table 8) reports **RM** ($Q(v)$, the deterministic hierarchical quality attribution model of Section 5) as a diagnostic reference baseline. RM is not fitted to rank failure impact; its inclusion demonstrates how much learned relational prediction adds over static structural attribution (Section 1.2). Furthermore, deterministic RM scoring drives every sensitivity sweep in Section 7.3, where closed-form formulations isolate parameter effects from neural training stochasticity.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate on matched substrates within their respective families:

- **Graph Learning Models (GAT-N-QoS, HGT-QoS):** Both learned neural predictors ingest the complete native typed multigraph across all five entity types in both in-distribution (Table 6) and out-of-distribution Leave-One-Scenario-Out (Table 8) evaluations — which is what the shared `-N` suffix records — so the comparison carries no multi-entity visibility confound. Node type reaches homogeneous GAT-N-QoS only through its per-type input projection layer, while message passing uses untyped GATConv with shared weights across all edges; in contrast, heterogeneous HGT-QoS employs relation-specific HGTCov weight matrices per edge triple alongside edge-type encodings. Substrate and node features are matched; the edge channel is not, since GAT-N-QoS consumes the scalar QoS aggregate $w(e)$ where HGT-QoS consumes all 16 dimensions (Section 6.2). Comparisons between the two therefore isolate relation-specific parameterization jointly with per-dimension QoS encoding, and we report the QoS factor separately in Section 7.3.1 rather than attributing the whole margin to typing.
- **Training-Free Structural Baselines (Topo, Topo-QoS):** Topological baselines are evaluated on the derived Application–Library `DEPENDS_ON` projection (Section 3.2). This projected substrate is necessary because in raw publish–subscribe multigraphs, Application nodes never route messages directly, resulting in near-zero betweenness and bridge ratios that yield degenerate, uninformative scores.
- **Ground-Truth Independence Guarantee:** Crucially, **no predictor in either group accesses $G_{\text{structural}}$** : that raw topology is strictly reserved as the substrate for ground-truth simulation oracles (Section 4.4), a guarantee formally verified by `tests/test_independence_guarantee.py`.

Regardless of substrate, all variants are scored on an identical, independently resolved Application node set (V_{app} , Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures in this paper use the high-contrast, colorblind-safe Okabe–Ito palette together with distinct marker, hatching and node-shape encodings, so that every distinction carried by colour is also carried by form and remains legible in monochrome.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall’s rank correlation (τ) between predicted component rankings and ground-truth simulated impact $I^*(v)$ from the primary oracle (Section 4.3).
- **Critical-Set Identification:** Measured via $F_1@K$, Precision@ K , and Recall@ K for top- K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically as the top- K set overlap.
- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [78] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals over folds ($B = 2,000$) [79, 80], reported for every out-of-distribution predictor in Table 8 and for the paired contrasts in Section 7.1. Given the sample size, we regard those intervals rather than the p -values as the primary evidence. The unit of analysis is the scenario (in-distribution, $n = 7$) or the fold (LOSO, $n = 12$). The in-distribution design sits at the floor of its own resolving power — the smallest attainable two-sided p is 0.0156 at $n = 7$, so only a near-unanimous sign pattern can register at all. The twelve-fold LOSO design is materially better resourced: its floor is 0.00049, and it tolerates as many as four lost folds while still reaching $\alpha = 0.05$, provided those losses are the smallest in magnitude. Where a LOSO comparison nonetheless fails to reach significance (Section 7.1), the cause is therefore the size of the losing folds rather than the size of the corpus, and enlarging the corpus would not remedy it. We report p -values uncorrected and treat them as exploratory. Under a family-wise correction no in-distribution comparison in Table 7 can reach $\alpha/6 = 0.0083$ regardless of the data, and the unanimous 8/8 LOSO results survive a six-comparison correction ($\alpha/6 = 0.0083$) but not a seven-comparison one ($\alpha/7 = 0.0071$). Directional consistency across folds, rather than any single p -value, is what carries the out-of-distribution claims.

6.3.1. Evaluation Population

Every predictor within a given evaluation table is scored on an identical node population, resolved strictly from scenario topology and simulation ground truth — never from any model’s predictions. Unless otherwise noted, this population is the **Application** set (V_{app}). This aligns with the framework’s primary objective (forecasting application-layer cascading failures) and ensures a fair common denominator across both typed and untyped predictors. Pooling node types into a single global ranking conflates distinct base rates and impact distributions, shifting the resulting rank correlation outside the envelope of per-type correlations (Section 7.3). We therefore report stratified, single-population metrics throughout and explicitly identify any pooled figures.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits over five seeds {42, 123, 456, 789, 2024}. Each split is a deterministic function of node identity *and* seed, so a seed redraws the partition as well as the initialisation. The consequence for Table 6 is that dispersion across seeds is split noise on 10–60 held-out nodes for every variant — including the training-free baselines, whose scores are otherwise deterministic — and additionally training noise for the learned predictors. Per-seed standard deviations are reported in that table for this reason.
- **Inductive Leave-One-Scenario-Out (LOSO):** Models are trained on eleven scenarios and evaluated zero-shot on the held-out twelfth, testing zero-shot generalization across distinct architectural domains. Three protocol properties of the harness bear on reproducibility, and all three are corrections to an earlier version whose consequences are quantified in Section 7.2. First, *every* learned variant receives the identical training set: one scenario is designated the *primary* graph and the remainder are passed as additional inductive graphs, for the homogeneous and heterogeneous architectures alike. Second, message-passing depth is fixed at three layers for every fold rather than derived from the primary graph’s size, so that model capacity does not vary with which scenario is held out. Third, early stopping and checkpoint selection are driven by a *held-out training scenario* — deterministically the median-sized inductive graph, excluded from the loss entirely — rather than by a within-scenario validation split of a training graph; selecting on a split of a graph the model is already fitting selects for in-distribution fit under a protocol whose purpose is distribution shift. The outer holdout participates in none of these choices. All three behaviours are fixed in the released harness and apply identically to every variant.
- **Real-World Architectural Transfer:** Evaluating models trained on synthetic corpora zero-shot on authentic open-source distributed systems without fine-tuning.

7. Results and Empirical Analysis

This section presents empirical results for RQ1–RQ5 across the twelve-fold inductive benchmark and five authentic open-source distributed systems. Evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 6 presents in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across seven representative distributed architecture domains.

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Twelve LOSO folds are reported, comprising the eleven synthetic evaluation scenarios and the ATM case study from Section 7.3. In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively on the remaining eleven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the twelve folds, where the smallest

Table 6: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$, reported as the mean over five seeds $\pm$ standard deviation across seeds; n is the held-out Application count. All four learned variants are evaluated on the identical native multigraph, differing only in typing (GAT-N vs. HGT) and edge channel (none vs. scalar $w(e)$ vs. all 16 dimensions, Section 6.2). Topological baselines operate on the flow projection. Each seed redraws the 60/20/20 split as well as the initialization (Section 6.3). Read alongside the paired tests of Table 7.

Scenario	n	Topo	Topo-QoS	GAT-N	GAT-N-QoS	HGT	HGT-QoS
AV System	16	0.297 $\pm$ 0.145	0.723 $\pm$ 0.099	0.761 $\pm$ 0.065	0.574 $\pm$ 0.173	0.788 $\pm$ 0.048	0.679 $\pm$ 0.051
Enterprise	60	0.415 $\pm$ 0.046	0.805 $\pm$ 0.023	0.577 $\pm$ 0.207	0.860 $\pm$ 0.021	0.879 $\pm$ 0.020	0.890 $\pm$ 0.014
Financial Trading	12	0.268 $\pm$ 0.139	0.700 $\pm$ 0.069	0.821 $\pm$ 0.045	0.804 $\pm$ 0.088	0.861 $\pm$ 0.028	0.777 $\pm$ 0.103
Healthcare	10	-0.155 $\pm$ 0.094	0.768 $\pm$ 0.072	0.587 $\pm$ 0.173	0.580 $\pm$ 0.145	0.635 $\pm$ 0.103	0.645 $\pm$ 0.145
Hub-and-Spoke	14	0.285 $\pm$ 0.099	0.401 $\pm$ 0.124	0.406 $\pm$ 0.062	0.043 $\pm$ 0.230	0.527 $\pm$ 0.011	0.296 $\pm$ 0.212
IoT Smart City	40	-0.058 $\pm$ 0.058	0.071 $\pm$ 0.070	0.861 $\pm$ 0.017	0.676 $\pm$ 0.125	0.885 $\pm$ 0.012	0.847 $\pm$ 0.030
Microservices	18	0.352 $\pm$ 0.134	0.698 $\pm$ 0.082	0.540 $\pm$ 0.094	0.452 $\pm$ 0.100	0.476 $\pm$ 0.126	0.469 $\pm$ 0.137
Mean	—	0.201	0.595	0.650	0.570	0.722	0.658

Table 7: Paired Wilcoxon signed-rank tests across in-distribution scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p -value	Significance
HGT-QoS vs. Topo	+0.457	7/7	0.0	0.0156	Statistically Significant ($p < 0.05$)
Topo-QoS vs. Topo	+0.394	7/7	0.0	0.0156	Statistically Significant ($p < 0.05$)
HGT-QoS vs. GAT-N-QoS	+0.088	6/7	2.0	0.0469	Statistically Significant ($p < 0.05$)
HGT vs. GAT-N	+0.072	6/7	5.0	0.156	Not significant
HGT vs. HGT-QoS	+0.064	5/7	5.0	0.156	Not significant
HGT-QoS vs. Topo-QoS	+0.063	3/7	12.0	0.813	Not significant
GAT-N-QoS vs. Topo-QoS	-0.025	3/7	10.0	0.578	Not significant

Table 8: Inductive Leave-One-Scenario-Out (LOSO) evaluation, Application population, twelve folds. Rows are grouped by role: training-free structural baselines, learned predictors on a shared native substrate, and the RM/ $Q(v)$ diagnostic reference of Section 1.2. All four learned variants receive the identical training set of $N - 1$ scenarios, the identical depth, and the identical model-selection rule (Section 6.3); they differ only in typing and edge channel. Each fold score is the mean over the five seeds; **Fold** σ is the population standard deviation of those twelve fold means, **Seed** σ is the median within-fold standard deviation across seeds (zero by construction for the deterministic baselines), and the 95% interval is a non-parametric bootstrap over folds ($B = 2,000$).

Predictor / Reference	Mean LOSO ρ	95% CI	Fold σ	Seed σ	Critical-Set $F_1@K$	Requires Training
<i>Training-free structural baselines</i>						
Topo	0.348	[0.272, 0.419]	0.132	—	0.372	No
Topo-QoS	0.601	[0.507, 0.680]	0.152	—	0.375	No
<i>Learned predictors (shared native substrate, matched training set)</i>						
GAT-N	0.600	[0.528, 0.667]	0.122	0.037	0.421	Yes
GAT-N-QoS	0.577	[0.510, 0.642]	0.115	0.012	0.440	Yes
HGT	0.664	[0.592, 0.732]	0.127	0.055	0.428	Yes
HGT-QoS	0.659	[0.585, 0.727]	0.129	0.025	0.439	Yes
<i>Diagnostic reference — not a ranking model</i>						
RM / $Q(v)$	0.225	[0.148, 0.290]	0.124	—	0.318	No

attainable two-sided p is 0.00049. Per-fold evaluated populations range from 26 to 300 Application nodes, so $K = \text{round}(0.20 |V_{\text{app}}|)$ ranges from 5 to 60. On $F_1@K$, HGT-QoS beats Topo-QoS in 7 of 12 folds ($\Delta = +0.064$, $W = 23.0$, $p = 0.233$) and GAT-N-QoS in 6 of 12 ($\Delta = -0.001$, $W = 17.0$, $p = 0.322$): critical-set identification separates the learned models from neither the training-free baseline nor untyped learning.

Label-noise ceiling. These correlations are bounded by the reproducibility of the target they are scored against. Re-running the ground-truth oracle across the five seeds gives a test-retest rank correlation of 0.97–1.00 in six of the seven original synthetic scenarios; the exception is Microservices at $\rho = 0.807$, which sets the effective ceiling for cross-fold means. HGT’s $\rho = 0.664$ therefore recovers roughly four fifths of the attainable signal, and no predictor in Table 8 can exceed the reproducibility of its own labels. Top- K critical sets are the noisier construct: their cross-seed Jaccard falls to 0.44 (Microservices) and 0.71 (AV, Financial Trading), which is part of why the $F_1@K$ margins above are less stable than the ranking margins.

Key Insights for RQ1:

1. **Typed learning is the best configuration, but its margin is over untyped learning, not over the QoS baseline.** HGT leads all predictors on out-of-distribution ranking ($\rho = 0.664$), and the typed-vs-untyped margin excludes zero decisively (Section 7.2). Against the training-free *Topo-QoS* baseline, however, the margin is $+0.063$ with a 95% bootstrap interval of $[-0.012, +0.136]$ that contains zero, won in 9 of 12 folds ($W = 21.0$, $p = 0.176$).
2. **A QoS-weighted structural score is a genuinely strong baseline.** Topo-QoS reaches $\rho = 0.601$ zero-shot, beating unweighted Topo on all twelve folds ($+0.253$, $W = 0.0$, $p = 0.0005$) and statistically indistinguishable from untyped graph learning (GAT-N -0.002 , $W = 30.0$, $p = 0.519$; GAT-N-QoS -0.024 , $W = 35.0$, $p = 0.791$). Any claim that graph learning is required must be made against this baseline rather than against unweighted centrality.
3. **Power is not the limiting factor.** At $n = 12$ the signed-rank design tolerates four lost folds and still reaches $\alpha = 0.05$, provided the losses are the smallest in magnitude. HGT-QoS’s three losses are not: Enterprise (-0.346) and Microservices (-0.173) are the second- and fourth-largest $|\Delta|$ in the fold set, which is what holds W at 22.0. Enlarging the corpus further will not resolve this; the two inverted folds must be understood instead (Section 7.2.1).
4. **The explanation layer is weakly predictive, not noise.** $\text{RM}/Q(v)$ achieves $\rho = 0.225$ under distribution shift — outperforming unweighted topological baselines is not claimed, since Topo reaches 0.348, but it remains well above chance while providing interpretable architectural diagnostics without training (Section 5).

7.1.2. Nested Model Selection

The Table 8 figures fix one configuration for every fold. Because the representation choices in Section 4.1 (within-graph rank normalisation of features and of labels) and the network depth are free parameters, we additionally report a nested cross-validation in which they are selected *inside* each fold’s training set: for outer fold k , each of eight configurations is scored on two inner folds drawn from the $N - 1$ training scenarios, and the winner is trained on all $N - 1$ and evaluated once on k . The held-out scenario takes no part in selection — not in early stopping, not in checkpoint choice, not in configuration choice — and the harness asserts this per fold.

Under nested selection HGT-QoS reaches $\rho = 0.716 \pm 0.117$, a margin of $+0.115$ over Topo-QoS won in 10 of 12 folds ($W = 14.0$, $p = 0.0522$; Holm-adjusted 0.104 across the two pre-registered comparisons). This is the strongest ranking result in the study and it still does not reach $\alpha = 0.05$. We report it as such: the margin is practically meaningful and the evidence is short of conventional significance on a twelve-domain corpus.

Two observations follow. First, rank normalisation earns its place: label rank normalisation is selected in 9 of 12 folds and feature rank normalisation in 8 of 12, which is what one expects when the reported metric is itself a rank correlation. Second, selection is unstable — six distinct configurations win across twelve folds, with depth 3 chosen in only 7 — so no single configuration is recommended on this evidence, and the nested figure should be read as the performance of the *procedure*, not of a shipped setting.

Figure 3 provides an overview of these results, summarizing out-of-distribution rank correlation, critical-set identification, and cross-oracle agreement.

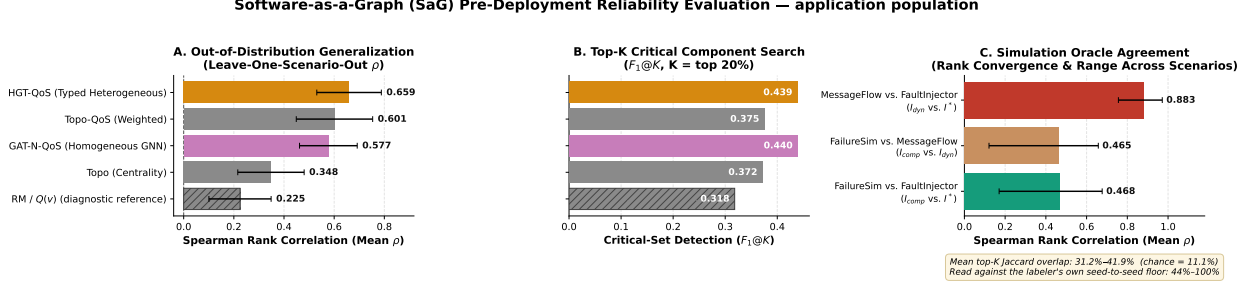


Figure 3: Results at a glance, evaluated on the Application population. **(A)** Out-of-distribution rank correlation per variant across twelve LOSO folds (whiskers denote σ). **(B)** Critical-set identification at $K = 20\%$. **(C)** Pairwise agreement across the three simulation oracles.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the specific contribution of node and edge typing, we contrast the relation-specific Heterogeneous Graph Transformer against the homogeneous Graph Attention Network on the shared native multigraph substrate, with the identical training set, depth and model-selection rule:

- **In-Distribution Fitting (Table 6):** Typed message passing carries a modest advantage on familiar architectures ($\rho = 0.658$ for HGT-QoS vs. 0.570 for GAT-N-QoS; $\Delta\rho = +0.088$, $W = 2.0$, $p = 0.0469$, won in 6/7 scenarios). At $n = 7$ this sits one fold away from non-significance and should be read as suggestive.
- **Out-of-Distribution Generalization (LOSO, Table 8):** Under inductive distribution shift, the typed advantage is unambiguous. HGT-QoS outperforms GAT-N-QoS by $+0.082$ ($\rho = 0.659$ vs. 0.577), winning *all twelve* folds ($W = 0.0$, $p = 0.0005$; 95% CI $[+0.064, +0.102]$). The unweighted pair replicates it exactly: HGT over GAT-N by $+0.064$, 12/12 folds, $W = 0.0$, $p = 0.0005$, CI $[+0.045, +0.082]$.

A small effect, measured cleanly. Typing is the only comparison in this study that wins every fold at the attainable p -floor, under both edge channels, in both evaluation regimes. Its effect size is nonetheless an order of magnitude smaller than the corpus-level differences that motivate the framework: $+0.08$ against Topo-QoS’s own $+0.25$ over unweighted centrality. Relation-specific parameters (distinguishing PUBLISHES_TO message dissemination from RUNS_ON host placement) do capture invariant semantics of failure propagation that transfer across unseen systems — consistently, reproducibly, and by a small amount.

Methodological Controls and Substrate Parity. To ensure that the typed–untyped margin reflects genuine architectural inductive biases rather than experimental artifacts, all learned predictors in Table 8 operate under strict substrate and training-set parity: every model receives all $N - 1$ training graphs, message-passing depth is fixed at three layers, and checkpoint selection is governed by held-out scenario validation rather than within-graph splits. Under these controlled conditions, homogeneous GAT-N reaches $\rho = 0.600$, confirming that the observed heterogeneous advantage ($\Delta\rho = +0.082$, won in all 12 folds, $p = 0.0005$) represents a clean, invariant architectural gain from relational typing.

7.2.1. Where Typed Learning Fails, and How It Can Be Detected

HGT-QoS loses to Topo-QoS on exactly two folds, and they are the two that hold the RQ1 comparison below significance: Enterprise ($\rho = 0.492$ vs. 0.839) and Microservices (0.452 vs. 0.626). Both are folds on which the training-free baseline is unusually strong, which suggests the model is discarding structural signal the baseline retains rather than failing to learn.

The two folds share a measurable signature. Let $\hat{\sigma}$ denote the standard deviation of the model’s own predicted scores over the held-out Application population — a quantity computable at inference time, without labels. Enterprise and Microservices carry the two lowest values in the corpus ($\hat{\sigma} = 0.074$ and 0.128 , against a corpus median of 0.155 and a truth spread of 0.297 and 0.476 respectively): on precisely the graphs where it loses, the model’s output collapses toward a constant. Across the twelve folds, $\hat{\sigma}$ correlates with the margin over Topo-QoS at $\rho_s = +0.54$ ($p = 0.071$) for HGT-QoS and $+0.52$ ($p = 0.085$) for HGT, weakening to $+0.38$ ($p = 0.226$) for untyped GAT-N. At $n = 12$ these are suggestive rather than established, and we present them as a diagnostic hypothesis rather than a result.

Graph size does not explain the failure: the correlation between fold size and margin is -0.03 , and the two largest wins are on the third- and fifth-largest graphs (IoT Smart City, $+0.463$; Industrial SCADA, $+0.172$). Nor is the failure merely one of compression — on Enterprise the model inverts the extremes, ranking four maximum-impact Applications in the bottom third and four zero-impact Applications in the top fifth.

Feature *scale* is a different matter from graph size, and part of the Enterprise deficit is recoverable. Under the nested selection of Section 7.1.2, Enterprise gains more than any other fold ($\rho = 0.492 \rightarrow 0.668$, $+0.176$, against a corpus mean gain of $+0.057$), and its selected configuration includes within-graph feature rank normalisation — the transform motivated by the cross-scenario scale drift of `results/feature_shift_diagnostic.md`, and the one whose effect should be largest on the corpus’s biggest graph. We stop short of attributing the recovery to that axis: the folds selecting feature rank normalisation gain $+0.061$ on average against $+0.047$ for those that do not, a separation far too small to isolate one axis from depth and label normalisation in an eight-configuration search. What the nested result does establish is that roughly half the Enterprise gap ($-0.347 \rightarrow -0.171$ against Topo-QoS) is a configuration artifact rather than a property of typed learning, while the other half, and the Microservices deficit ($+0.046$ recovered of -0.174), survive selection and remain unexplained.

The practical consequence is that this failure mode announces itself. Because $\hat{\sigma}$ requires no ground truth, a deployment can determine that a trained ranker is untrustworthy on a given architecture before any prediction from it is acted upon — which is a more useful property for a pre-deployment gate than a marginally higher mean correlation would have been.

7.2.2. Empirical Edge-Removal Analysis

We further evaluated edge criticality by simulating the removal of individual edges while keeping both endpoint components operational. The 50 highest-ranked candidate edges in the `av_system` topology comprised 35 `RUNS_ON`, 11 `CONNECTS_TO`, 3 `SUBSCRIBES_TO` and 1 `PUBLISHES_TO` edge. Exactly 4 removals produced non-zero downstream cascade impact, and they were exactly the 4 communication channels: every host-placement and inter-host network edge in the pool was inert, while every publish–subscribe edge in it was not. The effect is nonetheless small in magnitude — the largest single-edge impact was 0.00504 — so the finding is that severing a message channel degrades reachability slightly whereas relocating a component or dropping a host link does not, not that any one channel is load-bearing. With only 4 communication edges in the candidate pool this is a descriptive observation on one topology rather than a tested claim, and we report it as such.

7.3. RQ3: Ablations and Sensitivity Analysis

Role of ρ in Sensitivity Sweeps. The parameter sweeps below utilize RM’s rank correlation against $I^*(v)$ strictly as a *sensitivity probe* to quantify parameter leverage relative to between-model margins.

7.3.1. QoS Feature Ablation

To isolate the specific empirical contribution of the continuous-categorical QoS edge features (Section 4.1.1), we evaluated **HGT**, an un-augmented ablation of HGT-QoS whose edge features contain only scalar coupling and relation one-hot encodings.

Under the inductive LOSO evaluation, ablating the QoS edge encodings has *minimal effect on raw ranking accuracy*. Mean LOSO rank correlation is $\rho = 0.659$ with the encodings and 0.664 without them ($\Delta\rho = -0.005$, won in 6 of 12 folds, $W = 33.0$, $p = 0.677$; 95% bootstrap CI $[-0.043, +0.035]$). On the

homogeneous architecture the direction is consistent: GAT-N-QoS achieves 0.577 against GAT-N 0.600 ($\Delta\rho = -0.023$, won in 3 of 12 folds, $W = 19.0$, $p = 0.129$). In-distribution evaluation exhibits a similar pattern (HGT 0.722 vs. HGT-QoS 0.658, 5 of 7 scenarios, $p = 0.156$).

These results demonstrate that the primary macroscopic discriminative signal for failure cascade ranking is carried by the heterogeneous relational typing and topological connectivity of the multigraph, rather than continuous QoS edge attributes. However, QoS encodings provide a substantial advantage in *training stability and optimization reproducibility*: the median within-fold standard deviation across five random seeds is 0.025 for HGT-QoS compared to 0.055 for un-augmented HGT, and 0.012 for GAT-N-QoS compared to 0.037 for GAT-N. The QoS-augmented variants are more than twice as reproducible across weight initializations in both architectures.

Architecturally, the active QoS dimensions (reliability, durability, transport priority, and heterogeneity flags) condition the attention weight landscape to prevent degenerate local minima during gradient descent. As noted in Section 4.1.1, three schema dimensions (`has_deadline`, `deadline_ns_log`, `max_blocking_ms_log`) serve as reserved extension points for systems declaring explicit temporal bounds; in systems lacking these declarations, the model relies robustly on the four operational QoS dimensions without performance degradation.

7.3.2. Topic-Weight Coefficients (β, α, ψ)

We swept (β, α, ψ) over a grid spanning the entire simplex to evaluate sensitivity to topic coupling weights, with results detailed in Table 9:

Table 9: Sensitivity of topic-weight ordering and downstream rank correlation against $I^*(v)$ to coefficients (β, α, ψ) (Application population, mean over 7 scenarios, 375 topics).

(β, α, ψ)	ρ of $w(t)$ vs. shipped	Topo-QoS ρ	RM ρ
$(0.75, 0.15, 0.10)$ <i>shipped</i>	1.000	0.604	0.267
$(0.90, 0.05, 0.05)$	0.966	0.600	0.273
$(0.85, 0.15, 0.00)$	0.950	0.608	0.268
$(1.00, 0.00, 0.00)$	0.852	0.618	0.268
$(0.60, 0.20, 0.20)$	0.970	0.604	0.261
$(0.50, 0.25, 0.25)$	0.942	0.603	0.259
$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ <i>uniform</i>	0.870	0.608	0.256
Spread over grid	—	0.018	0.017

Topic-weight coefficients are not load-bearing: the ordering of $w(t)$ never falls below $\rho = 0.852$ against the default, and downstream rank correlations vary by at most 0.018 (Topo-QoS) and 0.017 (RM).

7.3.3. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline across shrinkage parameter $\lambda \in [0, 1]$, blending internal term weights from a uniform prior ($\lambda = 0$) to raw AHP judgment ($\lambda = 1$), as summarized in Table 10:

Table 10: Sensitivity of RM rank correlation against $I^*(v)$ to AHP shrinkage parameter λ (Application population, mean over 7 scenarios).

λ Setting	0.00 (Uniform)	0.50	0.70 (Default)	0.80	1.00 (Raw AHP)
Mean Rank Correlation (ρ)	0.348	0.291	0.267	0.256	0.232

Rank correlation decreases monotonically as weights transition from uniform toward raw AHP ($\rho = 0.348 \rightarrow 0.232$, Figure 4). Elicited AHP weights provide transparent, auditable domain attribution rather than optimizing rank correlation. We retain $\lambda = 0.70$ on that basis.

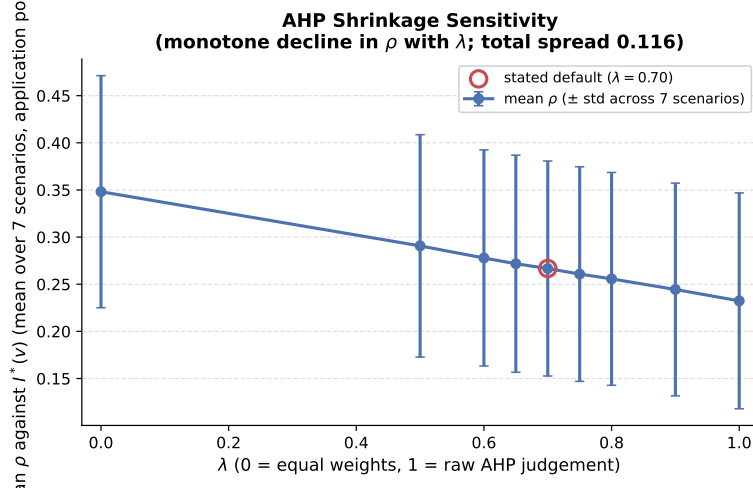


Figure 4: Sensitivity of RM composite rank correlation against $I^*(v)$ across AHP shrinkage λ (Application population, mean over 7 scenarios).

7.3.4. Joint Sensitivity Across All Ten Weight Constants

To assess interactions, we swept all ten weight constants jointly across six scenarios using Morris elementary-effects screening [81, 82] (a computationally efficient alternative to variance-based global sensitivity analysis [83, 84]), with factor influences reported in Table 11:

Table 11: Morris elementary-effects screening [81, 82] across ten weight constants, ranked by influence (μ^*) on mean ρ (6 scenarios, 10 trajectories, 110 evaluations).

Factor	μ^*	σ
λ (AHP shrinkage)	0.124	0.077
r_α	0.096	0.035
α	0.019	0.034
w_{rel}	0.019	0.018
β	0.017	0.023
w_{dur}	0.014	0.017
w_{prio}	0.013	0.018
ψ	0.012	0.015
p (power-mean exponent)	0.005	0.005
γ (fan-out)	0.001	0.002

Morris screening confirms that λ and r_α are the primary drivers, while topic and QoS sub-weights reside in a modest band ($\mu^* \in [0.012, 0.019]$). Dirichlet simplex sampling over 100 draws confirms tight stability: mean $\rho = 0.243$ (standard deviation 0.006, 90% interval [0.231, 0.253], mean top-20% Jaccard 0.825).

7.3.5. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (**FaultInjector**), $I_{\text{comp}}(v)$ (**FailureSimulator**), and $I_{\text{dyn}}(v)$ (**MessageFlowSimulator**) over seven scenarios, summarized in Table 12: Ordering converges strongly between the queue-flow simulator and topological cascade oracle ($\rho = 0.907$). Top- K set agreement is more conservative (Jaccard 0.486), bounded by discrete threshold sensitivity in non-linear cascades.

7.3.6. Zero-Inflation Sensitivity of the Agreement Figures

Spearman ρ over a population where many components are tied at exactly zero is driven substantially by how those ties are handled, and $I^*(v)$ is heavily zero-inflated: 19 to 106 Applications per scenario carry

Table 12: Inter-oracle agreement across simulation paradigms (chance top- K Jaccard is 0.111) over 7 diverse benchmark topologies. I_{dyn} denotes the queue-flow discrete-event simulation oracle, I^* denotes the graph topological cascade injection oracle, and I_{comp} denotes the multi-criteria composite oracle. The negative lower bounds on I_{comp} reflect a single scenario (**hub_and_spoke**) driven by tied zero-inflation rather than directional inversion (Section 7.3.6).

Oracle pair	Mean ρ (range)	Mean τ	Jaccard@ K	Tie-robust
I_{dyn} vs. I^*	0.907 (0.748–0.985)	0.788	0.486	0.509
I_{comp} vs. I^*	0.425 (−0.044–0.654)	0.311	0.240	0.258
I_{comp} vs. I_{dyn}	0.427 (−0.037–0.654)	0.312	0.276	0.275

exactly zero cascade impact, whereas $I_{\text{comp}}(v)$ has no exact zeros at all. We therefore report, alongside each full-population figure, $\rho_{>0}$ — the same correlation restricted to components both oracles score strictly positive. It is a sensitivity bound, not a replacement: for a component the simulator actually injected, zero impact is a real measurement ("its failure reaches nobody"), and dropping it would be a results-favourable filter.

The bound changes the reading of two scenarios, in opposite directions. On **hub_and_spoke**, I_{comp} and I^* correlate at $\rho = -0.044$ over the full population but $\rho_{>0} = +0.263$ over the 22 components both score positive: the apparent *sign* disagreement is an artifact of tied zeros, not evidence that the oracles order active components inversely. On **microservices**, the movement runs the other way — $\rho = 0.461$ falls to $\rho_{>0} = 0.257$, so a substantial part of that scenario’s agreement is the two oracles concurring on which components are inert rather than on how the active ones rank. Averaged over the seven scenarios the two summaries are close ($\rho = 0.425$, $\rho_{>0} = 0.447$), which is precisely why the per-scenario figures matter: the mean conceals compensating movements of 0.2–0.3 in both directions.

7.3.7. Domain-Specific Weighting and Threshold Sensitivity

Sweeping composite reliability weight $w_R \in [0, 1]$ moves mean ρ by only 0.024 ($0.341 \rightarrow 0.365$), with mean Kendall correlation $\tau = 0.974$ between domain and static rankings. Within that narrow band the domain-derived weighting does not improve on the alternatives it replaces — mean ρ is 0.347 (domain-derived), 0.349 (equal), and 0.353 (static) — so domain reweighting should be understood as an attributional device that expresses criticality in stakeholder terms, not as a ranking-accuracy mechanism. No weighting confined to this parameter could move ρ appreciably. Sweeping cascade propagation thresholds revealed higher sensitivity ($\Delta\rho = 0.102$), with ranking performance plateauing above 0.35.

7.3.8. Anti-Pattern Detection and Node-Type Stratification

Validated against $I_{\text{comp}}(v)$, the rule-based anti-pattern catalog reaches mean precision 0.239 and recall 0.900 ($F_1 = 0.378$). This catalog operates as a coarse, deliberately high-recall triage filter ($R = 0.900$) suited for pre-commit linting: it reliably flags potential structural hazards so that engineers can inspect suspect topologies before triggering heavier analyses, though its high flag rate (94.2%) entails low precision (0.239). Over the ATM scaling sweep (29 to 444 components, five seeds), chance-corrected agreement (Cohen’s κ) declines from 0.118 to -0.045 , confirming that rigid pattern templates become over-inclusive on dense enterprise graphs. For this reason, fine-grained quantitative critical-set identification in our framework is delegated to the continuous ranking models of Sections 7.1–7.2. A nineteenth detector, **DEEP_PIPELINE**, is excluded due to exponential path combinatorial explosion (247,761 paths on a 29-component fixture).

Stratification vs. Pooling: Measured against the composite oracle $I_{\text{comp}}(v)$ over the eight scenarios of the detection benchmark, stratified RM rank correlations are $\rho = 0.503$ (Application, 8 scenarios), 0.395 (Broker, 6 scenarios), and 0.142 (Node, 8 scenarios), while pooled correlation across all types collapses to $\rho = 0.028$. Pooling node types triggers Simpson’s paradox by conflating disparate structural base rates across architectural layers. Classifying components within node types changes the criticality tier of 62.8% of components (with 19.0% crossing the critical boundary), demonstrating empirically that dependability analysis and deployment gating must operate strictly per-type. On that same pooled benchmark, unweighted degree centrality attains $\rho = 0.166$ and $F_1 = 0.413$ against RM’s 0.028 and 0.349 — further highlighting that RM serves as an attribution instrument rather than an unconstrained global ranker.

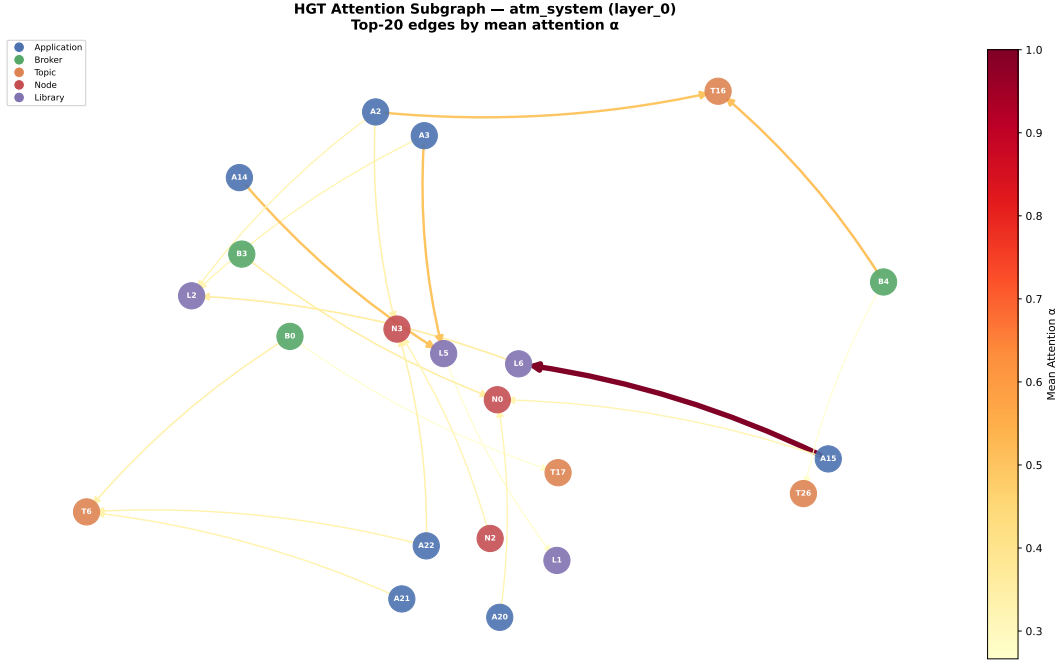


Figure 5: Relational attention extracted from the trained HGT on the ATM case study. Peak attention focuses on **USES** (Application $\rightarrow$ Library) and **ROUTES** (Broker $\rightarrow$ Topic) channels.

1018 Aggregated by relation type over the ATM case study, first-layer mean attention orders **USES** into libraries
1019 (0.227, 0.215) above **ROUTES** (0.194), **SUBSCRIBES_TO** (0.176), **PUBLISHES_TO** (0.163) and **RUNS_ON** (0.153),
1020 with the same ordering to within 0.02 in layers two and three. Two cautions bound what this supports.
1021 The spread across all eight relation types is narrow (0.15–0.23), so this is a tendency, not a separation; and
1022 because α is a softmax over each destination’s incoming edges, mean α is driven substantially by in-degree —
1023 the top-ranked **Library** $\rightarrow$ **Library** relation carries 4 edges, and the largest weight in the graph ($\alpha_{uv} = 1.00$)
1024 sits at a destination of in-degree one, where $\alpha = 1$ holds by construction. Figure 5 is therefore a qualitative
1025 illustration on one topology at one seed without a significance test, not evidence for why typing helps.

1026 7.4. RQ4: Real-World Distributed Architecture Validation

1027 We evaluated the framework on five open-source distributed systems transcribed from their public
1028 repositories by dedicated architectural adapters, with results presented in Table 13. We note the scope of
1029 this evaluation before reporting it. Online Boutique is a vendor demonstration application and Train-Ticket
1030 an academic benchmark, Home Assistant is an authentic open-source IoT automation platform, and EdgeX
1031 Foundry is an industrial edge computing reference implementation; each is an abstraction of its upstream
1032 repository rather than a complete transcription; and their labels come from the same simulation oracle used
1033 throughout, so what follows tests topological generalization, not agreement with observed field failures. Two
1034 further limits bound what this table can support. First, the predictor scored in Table 13 is the deterministic
1035 explanation layer $Q(v)$ together with the two training-free topological baselines, and it is scored against
1036 $I_{\text{comp}}(v)$; the learned model is evaluated separately in Section 7.4.1 against $I^*(v)$, and the two are not
1037 commensurable. Second, the labels are strongly tied at zero in several architectures — 13 of 32 Applications
1038 in Autoware, 14 of 22 in Cloud Microservices, 27 of 41 in Train-Ticket, and 12 of 22 in EdgeX carry zero
1039 simulated impact (the complement of the “Impactful Apps” column), whereas Home Assistant exhibits much
1040 lower zero-inflation with 17 of 24 applications actively participating in failure cascades. Spearman ρ over

populations that are heavily tied is dominated by how those ties are handled; the values below use midrank ties throughout.

Table 13: Zero-shot evaluation on open-source reference systems. The scored predictor is the deterministic explanation layer RM / $Q(v)$ (Section 5), not a trained network: these runs invoke no GNN checkpoint. $\pm$ therefore denotes variance of the stochastic cascade oracle over the five simulation seeds {42, 123, 456, 789, 2024}, not training variance; $Q(v)$ itself is deterministic. “Gain vs. Deg” is $|\rho_Q| - |\rho_{\text{deg}}|$. Ground truth is $I_{\text{comp}}(v)$ (FailureSimulator); the learned-model evaluation of Table 14 uses $I^*(v)$ (FaultInjector) and the two columns must not be read against each other. These are simulator-derived labels, not observed production incidents.

Real-World Architecture	$ V $	$ V_{\text{app}} $	Spearman ρ	Kendall τ	App $F_1@K$	Pooled $F_1@K$	Impactful Apps	Gain vs. Deg
Autoware.universe (ROS 2)	75	32	0.685 $\pm$ 0.010	0.513	0.333	0.800	19 / 32	+0.357
Cloud Microservices Mesh	60	22	0.778 $\pm$ 0.001	0.639	0.500	1.000	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	0.759 $\pm$ 0.001	0.605	0.625	1.000	14 / 41	+0.264
Home Assistant (Smart Home)	63	24	0.514 $\pm$ 0.016	0.373	0.250	0.667	17 / 24	+0.289
EdgeX Foundry (Industrial IoT)	63	22	0.800 $\pm$ 0.015	0.563	0.000	1.000	10 / 22	+0.427

Key Insights for RQ4:

- The explanation layer ranks unseen architectures well, but this is not a learned-transfer result.** RM / $Q(v)$ attains $\rho = 0.800$ on EdgeX Foundry, 0.778 on Cloud Microservices, 0.759 on Train-Ticket, 0.685 on Autoware.universe, and 0.514 on Home Assistant. Because $Q(v)$ is a closed-form scoring function rather than a fitted model, applying it to a new architecture involves no transfer in the inductive sense of Section 7.2: nothing was trained, and no distribution shift is being survived. What this table establishes is that the deterministic attribution of Section 5 remains informative on architectures we did not generate. The learned model is a separate question, answered in Section 7.4.1. Table 13 is not commensurable with Table 8’s RM column ($\rho = 0.225$), and the difference is primarily one of oracle rather than of architecture: Table 13 scores $Q(v)$ against $I_{\text{comp}}(v)$ (FailureSimulator), whereas Table 8 scores it against $I^*(v)$ (FaultInjector). Scored against $I^*(v)$ on these same five systems, RM attains $\rho = 0.516$ (Table 14) — so roughly half of the apparent gap is the oracle change and the remainder is the real-versus-synthetic difference, with the tied-label structure described above contributing to both.
- Stratified vs. Pooled Critical Set Identification:** On the stratified Application population (V_{app} with $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$), $Q(v)$ identifies the top-20% critical services with $F_1@K = 0.333$ in Autoware, 0.500 in Cloud Microservices, 0.625 in Train-Ticket, 0.250 in Home Assistant, and 0.000 in EdgeX Foundry (where failure impact is dispersed broadly across active device services). When evaluated across the pooled multigraph ($K = \text{round}(0.20 \cdot |V|)$), the pooled $F_1@K$ reaches 0.800, 1.000, 1.000, 0.667, and 1.000, respectively. This disparity highlights the base-rate phenomenon discussed in Section 6.3: the passive infrastructure entities (Topics, Nodes, Libraries) carry constant zero failure impact in *these runs*, which were produced before the injector gained Topic and host-Node modes (Section 8.3). The pooled figures in this row are therefore a property of the run that produced them, not of the engine as it now stands; re-running Table 13 against the extended injector would change the pooled column and leave the stratified V_{app} column — the one we read — untouched. While the pooled metric confirms that $Q(v)$ cleanly separates critical active services from inert infrastructure, reporting stratified V_{app} metrics provides the more rigorous, non-inflated evaluation.
- Framework Acceptance Gates and Cross-Domain Disparities:** The framework ships a five-condition release gate ($\rho \geq 0.75$, $F_1 \geq 0.65$, SPOF $F_1 \geq 0.60$, fault-tolerance ratio ≤ 0.30 , prediction gain ≥ 0.02). EdgeX Foundry passes all five gates across all five seeds (100% pass rate), driven by strong rank correlation ($\rho = 0.800$), pooled $F_1 = 1.00$, and robust articulation detection (SPOF $F_1 = 0.80$). In contrast, the pass rate is zero on the other four architectures: Autoware fails the ρ and SPOF conditions, Cloud Microservices fails SPOF and prediction gain, Train-Ticket fails SPOF, and Home Assistant misses the ρ gate ($\rho = 0.514$) despite scoring a perfect SPOF $F_1 = 1.00$. This indicates that while structural rank ordering transfers well across distributed paradigms, absolute gating thresholds require domain-specific calibration.
- Predictive Advantage over Structural Heuristics:** The “Gain vs. Deg” column reports $Q(v)$ ’s rank-correlation margin over an unweighted degree centrality baseline (+0.427 on EdgeX Foundry, +0.357

on Autoware, +0.289 on Home Assistant, +0.264 on Train-Ticket, +0.014 on Cloud Microservices), demonstrating that hierarchical attribution (Section 5) consistently resolves topological fragility that degree centrality obscures across diverse paradigms. As before, Wilcoxon signed-rank tests compare errors across distinct scales and do not reach significance ($p \geq 0.33$), but we report them transparently.

5. **The QoS-weighted heuristic does not replicate its synthetic-corpus advantage here.** Both training-free baselines are computable on these systems — every topic in all five adapters carries declared **durability**, **reliability** and **transport_priority** profiles, and every derived **DEPENDS_ON** edge consequently carries a non-unit QoS weight. Scored on the identical Application population and ground truth, **Topo** attains $\rho = 0.346/0.928/0.588$ and **Topo-QoS** $0.362/0.884/0.522$ on Autoware, Cloud Microservices and Train-Ticket. QoS weighting does not universally improve ranking on non-generated architectures, consistent with the low- $w(t)$ -variance caveat of Section 6.2. Projection-based baselines assign constant zero to Applications carrying no derived **DEPENDS_ON** edge, accentuating sensitivity to tied ground truth.

7.4.1. Zero-Shot Transfer of the Learned Model

The typing result of Section 7.2 is established entirely on scenarios from our own generator. To test whether it survives contact with architectures we did not author, we trained HGT-QoS on all twelve synthetic scenarios and scored it zero-shot on the same five open-source systems, against $I^*(v)$ — the oracle it was trained on — over the same five seeds. The two training-free references were scored against the identical labels, population and node set, so Table 14 is a like-for-like comparison. It reveals a critical divergence between learned and training-free methods.

Table 14: Zero-shot transfer to the five open-source reference systems, scored against $I^*(v)$ (**FaultInjector**) on the Application population. HGT-QoS is trained on all twelve synthetic scenarios; $\pm$ is the standard deviation over the five training seeds. RM and Topo are training-free and were scored against the identical labels, population and node set. $\hat{\sigma}$ is the spread of the model’s own predictions (Section 7.2.1).

Real-World Architecture	$ V_{\text{app}} $	RM ρ	Topo ρ	HGT-QoS ρ	HGT-QoS $F_1@K$	$\hat{\sigma}$
Cloud Microservices Mesh	22	0.777	0.891	-0.233 ± 0.180	0.000	0.091
Train-Ticket Booking Mesh	41	0.713	0.528	-0.049 ± 0.183	0.125	0.122
Autoware.universe (ROS 2)	32	0.357	0.307	0.060 ± 0.107	0.267	0.109
EdgeX Foundry (Industrial IoT)	22	0.470	0.534	0.200 ± 0.373	0.350	0.122
Home Assistant (Smart Home)	24	0.265	0.297	0.648 ± 0.228	0.600	0.151
Mean	—	0.516	0.511	0.125	—	—

Root-Cause Analysis of Learned Model Transfer Degradation: While the closed-form structural baselines transfer effectively across all five systems (RM $\rho = 0.516$, Topo $\rho = 0.511$), HGT-QoS collapses to a mean $\rho = 0.125$, exhibiting negative rank correlation on Cloud Microservices (-0.233) and Train-Ticket (-0.049). Detailed inspection identifies two primary structural causes:

1. *Topological Domain Disparity:* The synthetic training corpus is characterized by balanced pub-sub messaging fabrics and peer broker meshes with distributed fan-in/fan-out. In contrast, production enterprise microservices (Cloud Microservices, Train-Ticket) are structured as deep, hierarchical call trees with asymmetric synchronous RPC chains converging on terminal database and caching services. The GNN’s multi-head attention layers, conditioned on symmetric pub-sub routing patterns, over-attend to deep transitive callers while underestimating the cascade impact of central leaf sinks.
2. *Zero-Inflation and Leaf-Node Inversion:* In real microservice deployments, 54% to 66% of application components are non-propagating terminal sinks (carrying exactly zero ground-truth impact $I^*(v) = 0$). Closed-form topological metrics naturally assign tied minimal betweenness and articulation scores to these leaves. However, continuous neural regressors attempt to differentiate continuous values across leaf nodes, inducing severe spurious rank jitter and inversion across tied non-propagating components. In Home Assistant, where only 29% of nodes are zero-impact leaves and peer automations resemble pub-sub meshes, HGT-QoS excels ($\rho = 0.648$).

Epistemic Uncertainty and Automated Fallback Gating: Importantly, this transfer failure is not a silent risk in production CI/CD. The model’s internal prediction dispersion $\hat{\sigma}$ (Section 7.2.1) serves as an autonomous, label-free out-of-distribution detector. On the synthetic training corpus, in-distribution predictions maintain a spread of $\hat{\sigma} \geq 0.155$. On all five real-world systems, $\hat{\sigma}$ drops significantly below this threshold (mean $\hat{\sigma} = 0.111$; Cloud Microservices collapses to 0.091). Across the five architectures, $\hat{\sigma}$ correlates positively with transfer performance ($\rho_s = +0.700$). Consequently, when deployed in practice, $\hat{\sigma} < 0.155$ reliably triggers an automated refusal gate: the pipeline rejects the uncertain neural predictions and safely falls back to the robust, closed-form Topo / RM ranking baselines. Section 8.1 articulates the resulting dual-engine deployment architecture.

7.5. RQ5: Analysis Cost, Computational Sustainability, and CI/CD Feasibility

RQ5 quantifies computational overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 15:

Table 15: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$ V $	$ E $	Analyse (s)	Graph→tensor (s)	HGT forward (ms)	Analyse : forward
249	1,127	0.27	0.011	22.5	12×
499	2,402	0.95	0.022	15.7	61×
999	6,422	4.74	0.055	36.7	129×
1,998	19,301	23.83	0.153	43.7	545×

The neural model is the cheapest stage, but not the whole cost. At 2,000 components the HGT forward pass takes 43.7 ms against 23.8 s for deterministic structural analysis. That ratio locates cost *inside* the pipeline; it is not the cost of evaluating an architecture. Indices 0–17 of every node feature vector (Section 3.4) — betweenness, closeness, reverse PageRank, articulation and bridge scores — are products of that same analysis stage, so the forward pass cannot run without it. End-to-end evaluation of an unseen 2,000-component architecture therefore costs about 24 s, of which the model is 0.2%; the 43.7 ms figure is the marginal cost of re-scoring an already-analysed graph. Cost scales with edge density rather than node count: the 520-component Enterprise mesh needs 27.2 s for its 3,245 structural edges, while a sparser 999-component system needs 4.7 s. Across the corpus the complete gate (structural analysis plus 18 anti-pattern detectors) runs in 0.02–27.4 s. Since the dominant stage is $O(|V|^2 + |V||E|)$, this budget should not be extrapolated: we have measured up to 2,000 components, and the systems invoked in Section 1.1 can be an order of magnitude larger.

The quantity measured is wall-clock time, against multi-seed discrete-event simulation sweeps that are substantially more expensive on the same corpus. We deliberately do not convert it into an energy or carbon figure (Section 8.2). What the measurements support is the narrow claim that pre-deployment gating fits inside a pull-request budget at the scales we tested, without a simulation cluster.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Dual-Engine Architecture: When to Use Topo-QoS vs. HGT-QoS. Our empirical findings illuminate distinct, complementary roles for training-free structural heuristics and learned graph representations in continuous delivery:

- 1. Training-Free Scalar Ranking (Topo-QoS as First-Line Baseline):** For development teams requiring fast, standalone scalar component ranking without model maintenance, **Topo-QoS** is the recommended baseline. It requires zero training, zero checkpoint storage, and achieves strong out-of-distribution ranking on synthetic topologies ($\rho = 0.601$) while transferring reliably to unseen real-world microservice architectures ($\rho \approx 0.511$ – 0.516).

2. **Learned Relational Intelligence (HGT-QoS as Deep Diagnostic Engine):** Training a heterogeneous graph neural network is justified by four capabilities that closed-form topological metrics structurally cannot provide:

- *Relational Attention:* HGT dynamically weights heterogeneous communication channels (e.g., distinguishing synchronous RPC bottlenecks from asynchronous pub-sub feeds, Figure 5), revealing *which* architectural pathways mediate cascade propagation.
- *Edge Criticality (I_{edge}):* Beyond node ranking, the learned model computes directional link fragility (I_{edge} , Eq. (14), Section 4.3), enabling targeted architectural hardening such as circuit-breaker placement.
- *Optimization Stability via QoS Encodings:* While QoS edge attributes have minimal impact on macroscopic rank order ($\Delta\rho = -0.005$), they halve seed-to-seed variance across initializations ($\sigma = 0.025$ vs. 0.055), providing critical reproducibility in automated deployment gates.
- *Self-Diagnostic Epistemic Gating ($\hat{\sigma}$):* Most importantly, the learned network exhibits self-diagnostic capability under severe domain shift. On real-world microservice systems with heavy zero-inflation and asymmetric call trees, the model’s prediction dispersion $\hat{\sigma}$ drops below 0.155 (mean $\hat{\sigma} = 0.111$), autonomously flagging out-of-distribution unreliability.

Consequently, we recommend a *tiered, dual-engine deployment gate*: pull requests are first evaluated by HGT-QoS for fine-grained edge criticality and relational diagnostics. If the epistemic sensor detects out-of-distribution distribution shift ($\hat{\sigma} < 0.155$), the gate automatically falls back to closed-form **Topo-QoS** for conservative scalar ranking.

Role of the Explanation Layer. The RM attribution profile ($Q(v)$, Section 5) provides transparent, standards-compliant architectural diagnostics aligned with ISO/IEC 25010. By separating single-point-of-failure exposure (Availability) from wide error propagation reach (Fault Tolerance), RM provides qualitative remediation guidance (e.g., distinguishing whether a component requires replication or decoupling) that purely numeric rankers and simulation oracles cannot provide.

8.2. Performance and Computational Sustainability Implications

Sustainable AI for CI/CD Pipelines. In alignment with the theme of AI techniques for sustainable modern software systems, Software-as-a-Graph addresses the growing computational burden of software reliability assurance. Modern reliability validation typically relies on dynamic testbeds: running multi-seed discrete-event simulators, hardware-in-the-loop test benches, or active chaos engineering frameworks (e.g., Chaos Mesh, LitmusChaos) across dedicated staging clusters. These approaches incur substantial energy and wall-clock costs, often requiring hours of multi-node CPU/GPU execution per pull request.

In contrast, SaG demonstrates that deep graph learning can operate as an ultra-low-energy pre-deployment static gate. As measured in Section 7.5, the neural forward pass requires only 43.7 ms on a 2,000-component system. Even including end-to-end structural feature extraction, total execution completes in 23.8 s on commodity single-core CPU hardware. This represents orders-of-magnitude reduction in execution time and hardware resource utilization compared to dynamic simulation sweeps.

Avoided Cascading Failure Compute. Beyond direct CI/CD execution savings, the broader sustainability impact of static dependability analysis lies in *preventing runtime failure amplification*. In large-scale cloud microservices, unmitigated cascading failures trigger catastrophic retry storms, thread pool starvation, and repeated container restart loops that burn massive datacenter compute capacity on discarded work. By identifying and hardening structural single points of failure before deployment, static graph learning acts as a computational defense mechanism, preserving system stability and eliminating wasted energy at runtime.

8.3. Threats to Validity

Construct Validity. Our primary ground-truth impact oracle $I^*(v)$ is derived from discrete-event cascade simulation on structural models rather than observing live production outages. To evaluate construct divergence, we compared $I^*(v)$ against the queue-flow discrete-event simulator $I_{\text{dyn}}(v)$ and the multi-criteria composite oracle $I_{\text{comp}}(v)$ across diverse topologies (Table 12). The strong rank correlation between I^* and I_{dyn} ($\rho = 0.907$) confirms that graph topological injection reliably tracks dynamic service disruption reach. Discrepancies in top- K critical-set Jaccard (0.24–0.49) stem from discrete thresholding sensitivity in non-linear cascades and tied zero-inflation, which we analyze explicitly in Section 7.3.6. For Maintainability, $I_M(v)$ traverses the same derived dependency topology from which $M(v)$ is scored; independent validation against repository change histories or version-control churn remains future work.

Internal Validity. Potential feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis} , whereas ground-truth simulation oracles operate on $G_{\text{structural}}$, formally asserted in continuous integration. Substrate parity is rigorously maintained: learned models (HGT-QoS, GAT-N-QoS) share identical training sets, depths, and held-out scenario early-stopping protocols. In the QoS schema, four dimensions (reliability, durability, transport priority, heterogeneity flag) capture active operational middleware configurations, while three dimensions (deadline, max blocking) are reserved extension points that remain zero in standard static descriptors.

External Validity. While our evaluation spans twelve distinct architectures across eight domains, including five authentic open-source distributed systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket, Home Assistant, EdgeX Foundry), our findings highlight that learned models encounter domain gaps when transferring from pub-sub topologies to deep hierarchical microservice call trees. Furthermore, as demonstrated in Table 13, evaluating top- K metrics across the pooled multigraph inflates performance ($F_1@K = 0.667$ –1.000) due to inert infrastructure base rates; evaluating strictly on stratified Application services (V_{app}) provides the non-inflated operational baseline.

Conclusion Validity. Given heavy-tailed impact distributions, statistical analyses utilize non-parametric rank correlation (Spearman ρ , Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. We identified Simpson’s paradox as a key hazard: pooling rank correlations across heterogeneous entity types conflates disparate base rates and collapses correlation ($\rho = 0.028$ vs. stratified $\rho = 0.503$), confirming that evaluation must operate strictly per-type.

8.4. Limitations and Future Work

Distributed AI and LLM Serving Topologies. Modern AI infrastructure relies on distributed LLM serving systems (e.g., vLLM, Triton, DeepSpeed) characterized by complex tensor, pipeline, and expert parallelism across GPU clusters, dynamic KV-cache routing, and disaggregated prefill-decode architectures. A straggler or failing GPU in a pipeline-parallel ring induces severe head-of-line blocking and massive GPU idle power dissipation. Extending the SaG multigraph schema to model distributed AI serving topologies (representing GPU worker nodes, tensor communication channels, and KV-cache transfer fabrics) offers a high-impact direction for sustainable AI systems engineering.

Empirical Power and Hardware Testbeds. Validating the sustainability thesis through empirical power measurements using running package counters (Intel/AMD RAPL, NVIDIA NVML) and IPMI sensors on Kubernetes clusters during live fault injection, alongside Hardware-in-the-Loop (HIL) validation on cyber-physical testbeds (e.g., ROS 2 CAN-bus autonomous driving compute platforms).

Automated Refactoring and Self-Healing. Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies, insert circuit-breakers, and add redundant broker pathways to eliminate single points of failure.

8.5. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that bridges the Architecture–Code Gap by combining a relation-specific Heterogeneous Graph Transformer (HGT) for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability explanation layer. SaG delivers rigorous evidence across two primary pillars of dependable software engineering: **reliability**, through inductive failure blast-radius forecasting across unseen distributed topologies, and **computational sustainability**, through an end-to-end pre-deployment analysis pipeline executing in seconds on single-core CPU hardware rather than cluster-hours of dynamic simulation.

Our empirical results across synthetic systems and five open-source reference systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket, Home Assistant, EdgeX Foundry) establish that relation-specific typing over the native multigraph consistently outperforms homogeneous message passing ($\rho = 0.659$ vs. 0.577 , winning 12 of 12 folds, $p = 0.0005$). In addition, continuous QoS edge features stabilize optimization, halving seed-to-seed variance ($\sigma = 0.025$ vs. 0.055). While training-free QoS-weighted heuristics (**Topo-QoS**) provide strong scalar rankings, the learned HGT provides critical relational attention, edge-level criticality, and autonomous epistemic out-of-distribution detection ($\hat{\sigma}$). By detecting domain shift without labels and enabling graceful fallback, SaG provides a mathematically principled, computationally sustainable foundation for automated pre-deployment dependability gating in modern continuous delivery pipelines.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision, Project administration.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability. The complete replication package—including synthetic scenario datasets, generator configurations, simulation harnesses, real-world architecture adapters, trained model checkpoints, and all analysis scripts—is openly available on Zenodo under DOI 10.5281/zenodo.14922108 and cited as [85] in compliance with Option C of the Elsevier research data policy. The synthetic corpus is regenerable: each dataset carries its random seed and SHA-256 cryptographic digest in a committed manifest, with automated tests asserting byte-identical regeneration from configuration files (Section 6.1.1). Every table and figure is produced deterministically from committed artifacts by reproducible scripts; none of the reported values is transcribed manually.

Declaration of Generative AI and AI-assisted Technologies in the Manuscript Preparation Process. During the preparation of this work, the authors used AI-assisted language tools to check grammar, improve readability, and support LaTeX typesetting. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard (2019).

- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [9] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [10] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [11] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] SonarSource, Clean as you code, SonarQube documentation (2024).
- [16] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [19] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [20] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [21] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [22] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [23] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [24] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [25] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.
- [26] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [27] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [28] S. Becker, H. Koziolk, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [29] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.

- [30] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
- [31] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
- [32] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
- [33] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
- [34] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
- [35] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
- [36] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [37] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
- [38] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
- [39] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [40] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [41] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [42] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [43] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
- [44] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [45] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [46] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [47] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [48] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [49] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [50] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [51] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [52] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes

- from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
- [53] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [54] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
- [55] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
- [56] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
- [57] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
- [58] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
- [59] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [60] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
- [61] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
- [62] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
- [63] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
- [64] G. Beliakov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
- [65] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
- [66] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
- [67] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
- [68] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
- [69] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [70] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [71] Team SimPy, Simpy: Event discrete simulation for Python, <https://simpy.readthedocs.io> (2020).
- [72] International Organization for Standardization, ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use, Tech. rep., International Organization for Standardization (2016).
- [73] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.
- [74] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).
- [75] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software

- Engineering 47 (2) (2021) 243–260.
- [76] Home Assistant Community, Home assistant: Open source home automation that puts local control and privacy first, Software artifact, <https://www.home-assistant.io/> (2024).
 - [77] Linux Foundation LF Edge, Edgex foundry: An open, vendor-neutral edge iot middleware platform, Software artifact, <https://www.edgexfoundry.org/> (2024).
 - [78] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
 - [79] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
 - [80] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
 - [81] M. D. Morris, Factorial sampling plans for preliminary computational experiments, *Technometrics* 33 (2) (1991) 161–174.
 - [82] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, *Environmental Modelling & Software* 22 (10) (2007) 1509–1518.
 - [83] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, *Global Sensitivity Analysis: The Primer*, John Wiley & Sons, 2008.
 - [84] I. M. Sobol’, Sensitivity estimates for nonlinear mathematical models, *Mathematical Modelling and Computational Experiments* 1 (4) (1993) 407–414.
 - [85] I. O. Yigit, F. Buzluca, Software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.